

重庆大学计算机专业核心课程《机器学习》

第6章 支持向量机

授课教师：郑林江

引言：为什么我们需要支持向量机？

机器学习中的分类挑战：从线性可分到现实复杂数据

增强模型鲁棒性

鲁棒性确保模型在扰动下保持性能：需减少对局部变化的敏感度，提高对未知样本的适应力。

多决策边界选择

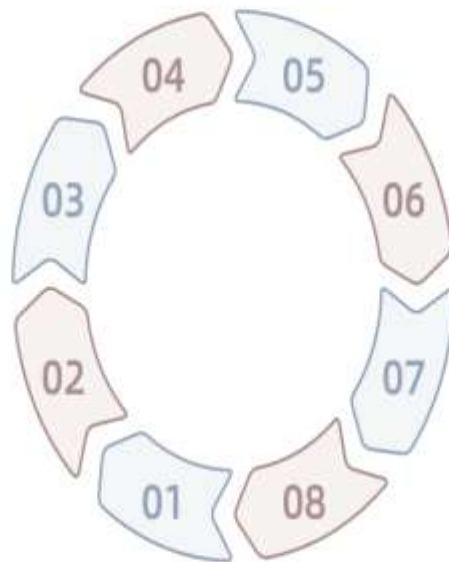
存在多个可行的划分边界，泛化能力不同：需评估边界合理性，选择更具稳定性的方案。

应对线性不可分

当特征无法被直线划分时，需引入非线性方法：如核技巧或复杂边界模型，以更好拟合实际分布。

处理数据噪声

现实数据常包含噪声和重叠，导致线性不可分：需通过预处理或鲁棒方法降低干扰，提升模型对异常值的容忍度。



边界泛化能力

决策边界应具备良好的外推能力，避免过拟合训练数据，提升在测试集上的表现稳定性。

异常点稳定性

模型应抵抗异常点的影响，防止个别样本扭曲决策面，保障整体分类逻辑一致性。

数据分布扰动

真实场景中数据分布可能变化，模型需适应分布偏移，保持预测结果的一致性和可靠性。

优化边界合理性

合理边界应平衡间隔与误差，考虑全局结构而非局部细节，提升整体泛化与稳定性表现。

SVM的定位：小样本下的高泛化能力模型



与其他线性分类器的对比：逻辑回归 vs 感知机 vs SVM

分类器	决策机制差异	损失函数设计	解的稀疏性	泛化能力对比
逻辑回归	输出概率	对数损失 考虑所有样本 置信度	使用全部数据	易受极端值影响
感知机	仅求可分	感知机损失 只关注分错的 样本	使用全部数据	无唯一解
SVM	追求最大间隔, 兼顾准确性与 鲁棒性	hinge损失 关注分错 + 边界附近的样 本	仅依赖支持向 量构建模型, 预测高效	通过最大间隔 提升泛化性, 小样本下表现 更稳

SVM的发展脉络与在现代AI中的角色



起源与奠基

SVM由Vapnik等人于1995年正式提出，基于统计学习理论和结构风险最小化原则，奠定了小样本高泛化模型的基础。



历史转折点

20世纪末至21世纪初，SVM在文本分类、手写识别等任务中超越传统方法，成为机器学习主流算法之一。



现代AI中的定位

虽被深度学习部分取代，但在小样本、高维稀疏数据场景中仍具独特优势与不可替代性。

第6章 支持向量机

➡ 6.1 间隔与支持向量

6.2 对偶问题

6.3 核函数

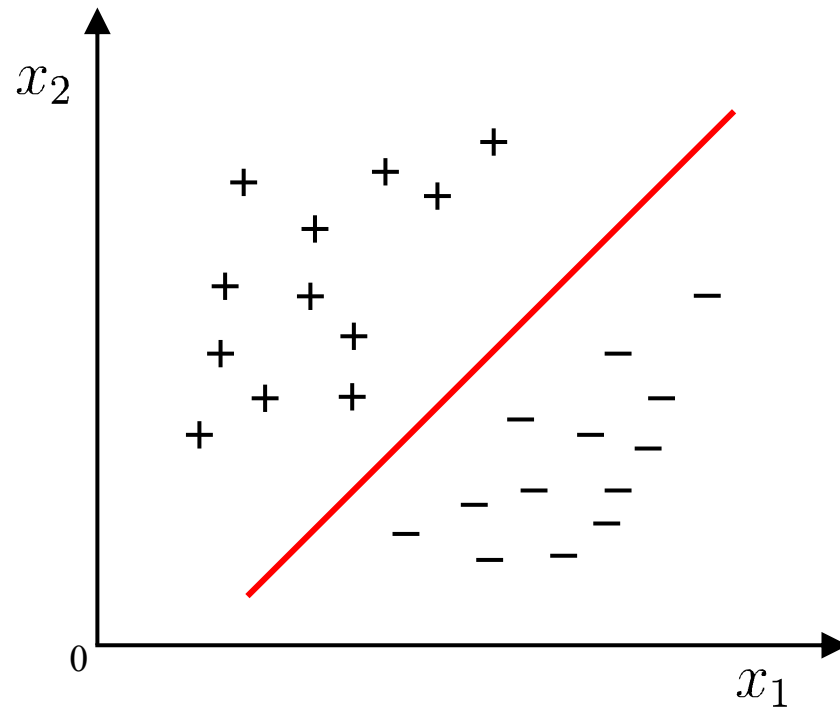
6.4 软间隔与正则化

6.5 支持向量回归

6.6 核方法

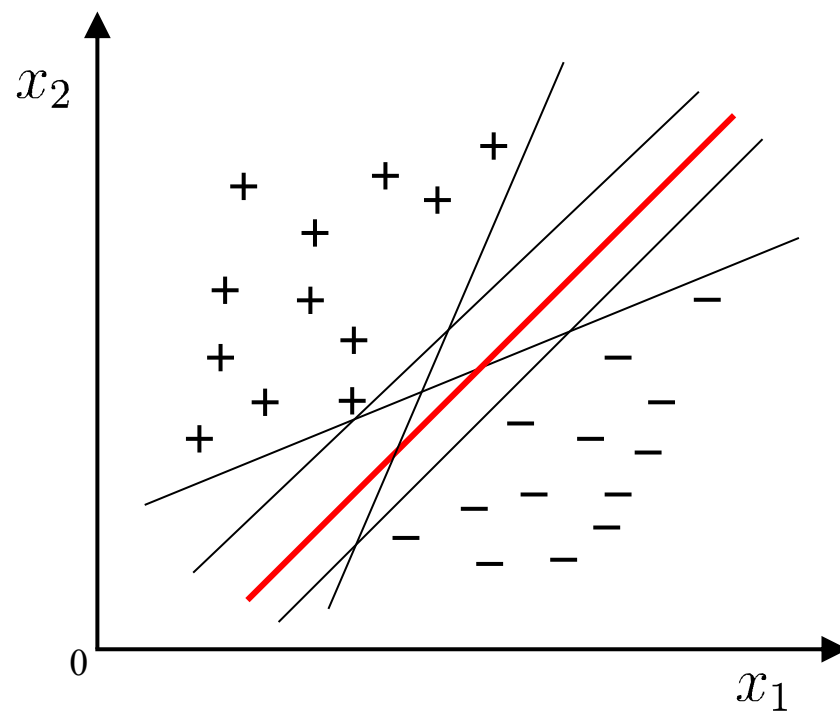
引子

线性模型：在样本空间中寻找一个超平面，将不同类别的样本分开。



引子

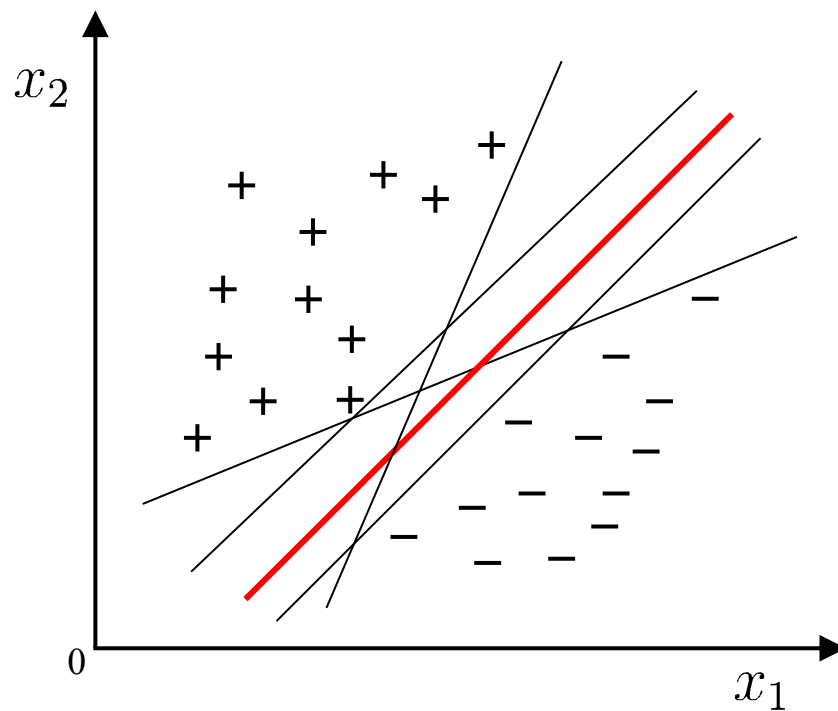
-Q: 将训练样本分开的超平面可能有很多, 哪一个好呢?



存在多个划分超平面将两类训练样本分开

引子

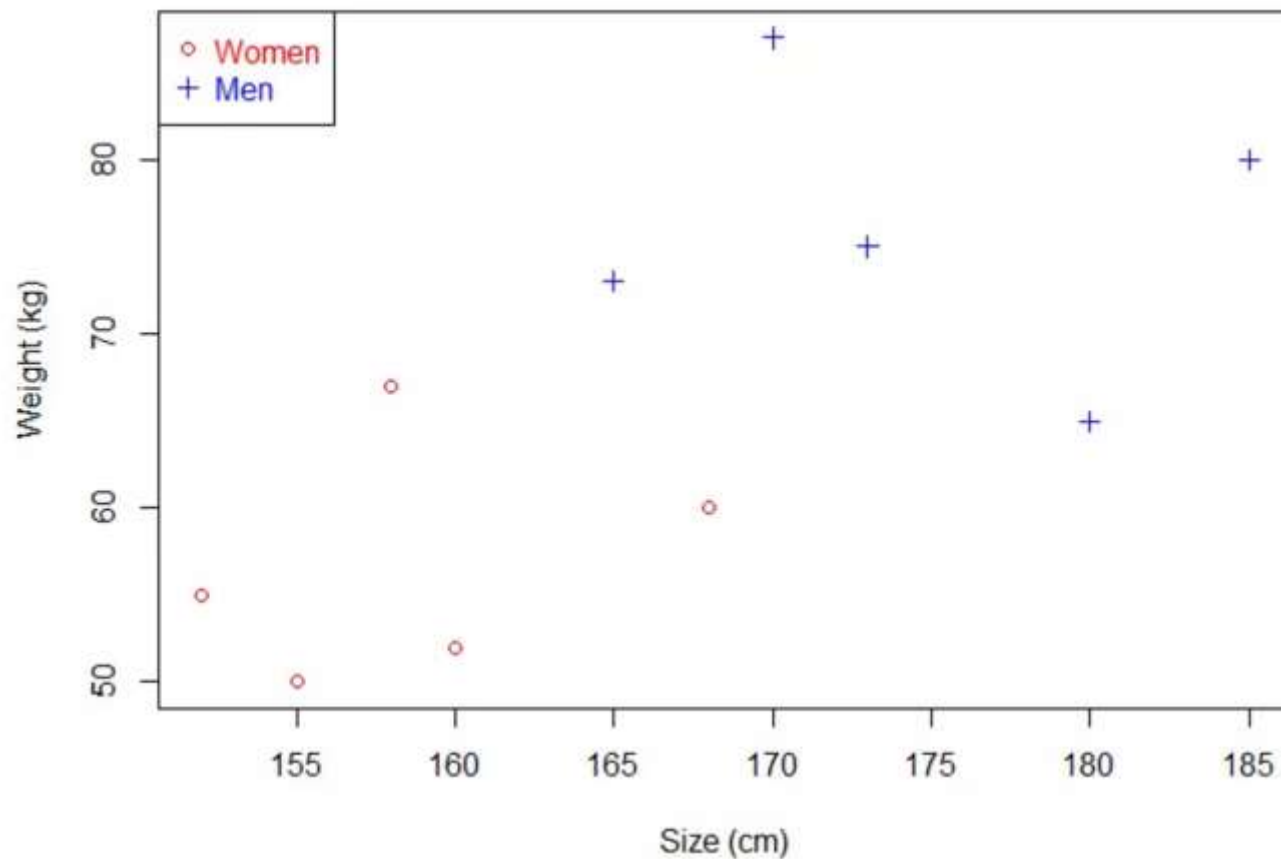
-Q: 将训练样本分开的超平面可能有很多, 哪一个好呢?



-A: 应选择” **正中间**”, 容忍性好, 鲁棒性高, 泛化能力最强.

基本原理

- 某人高175厘米，重80公斤。
- 是男性还是女性呢？



分离超平面

□ 画一条线，使得

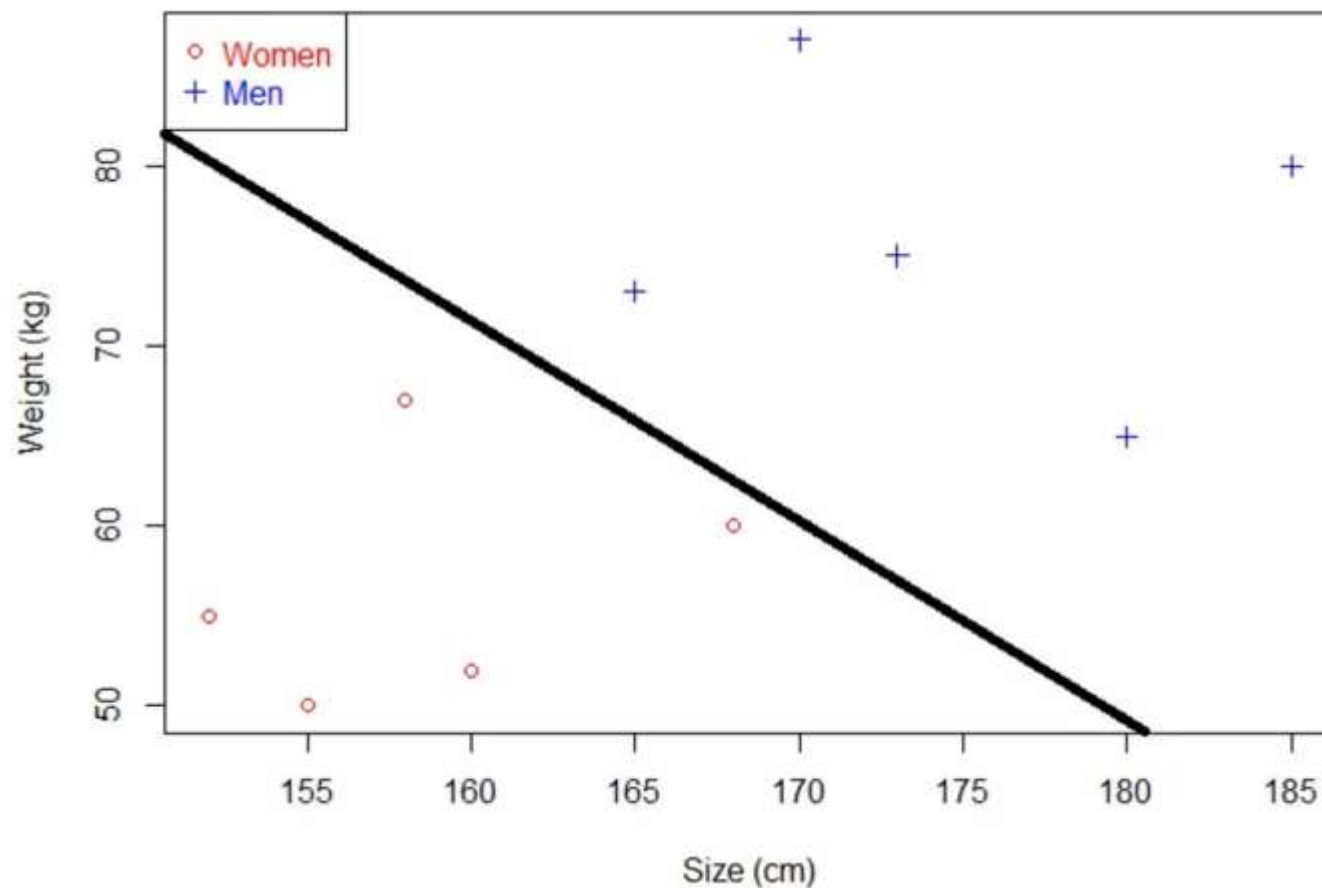
所有的男性数据
点在直线上方，

所有的女性数据
点在直线的下方。

□ 在逻辑回归当中，

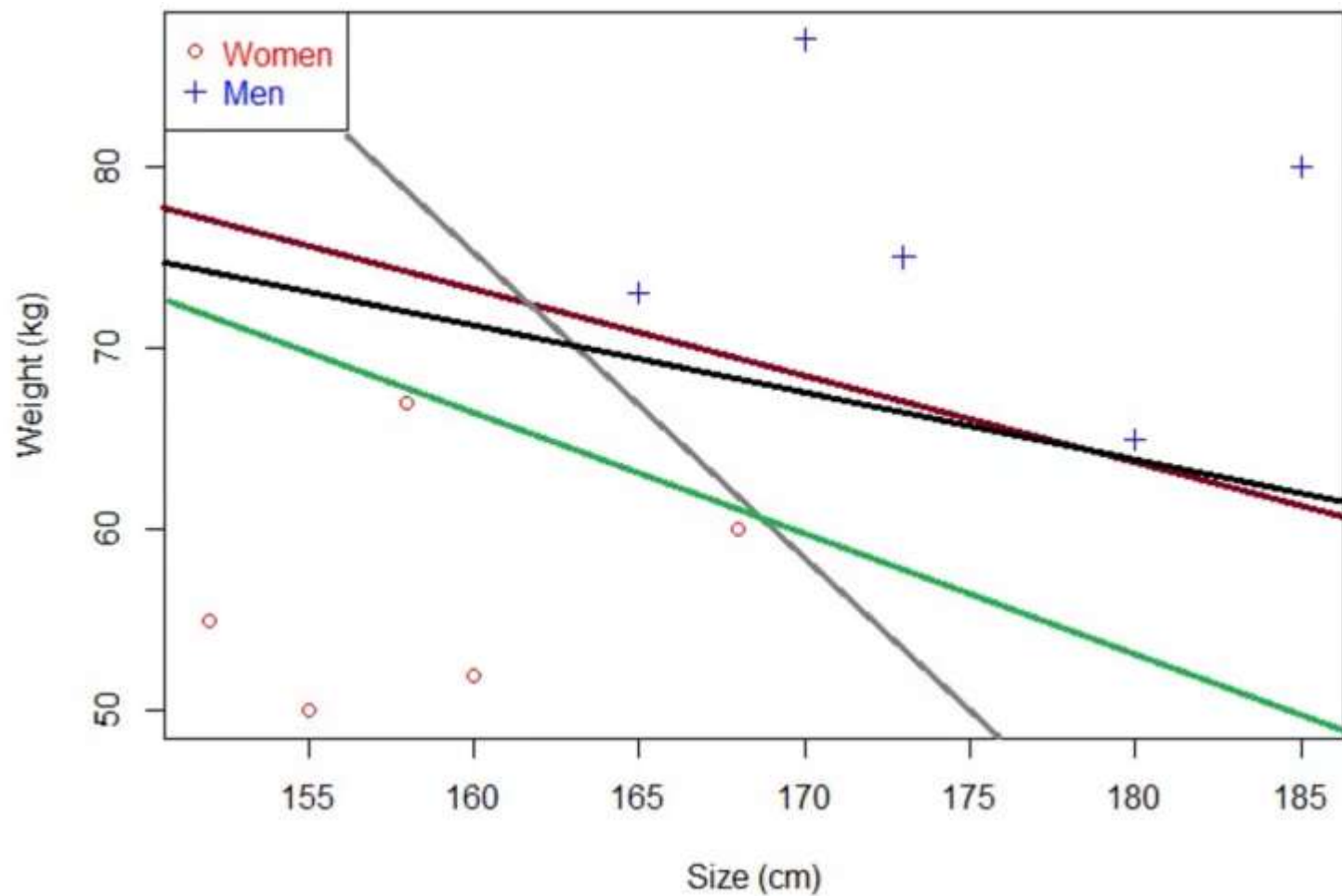
这条直线叫做决
策边界，在SVM当

中叫做**分离超平**
面。



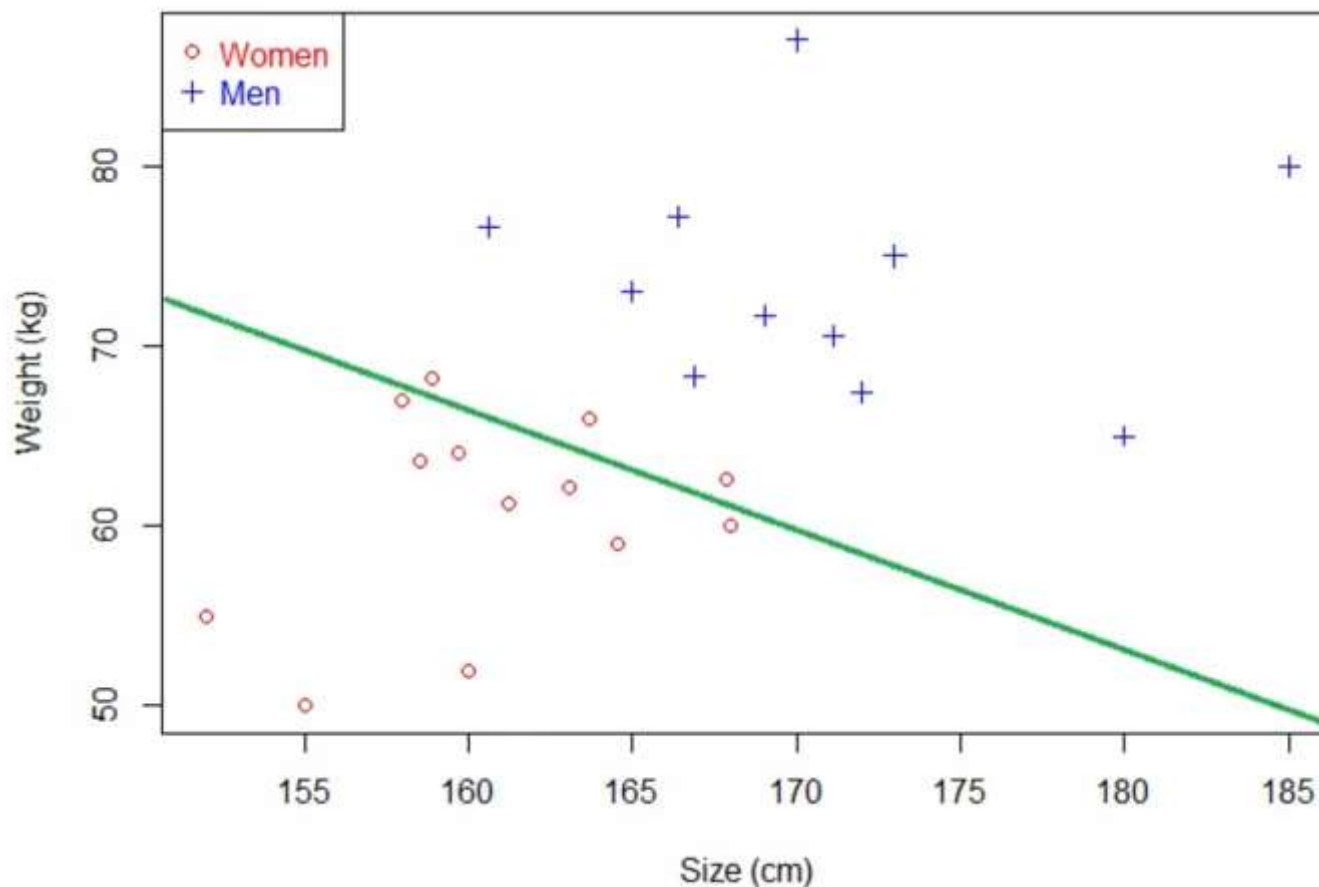
最优分离超平面

□ 几个分离超平面都是正确的，但是不是分类效果不同。



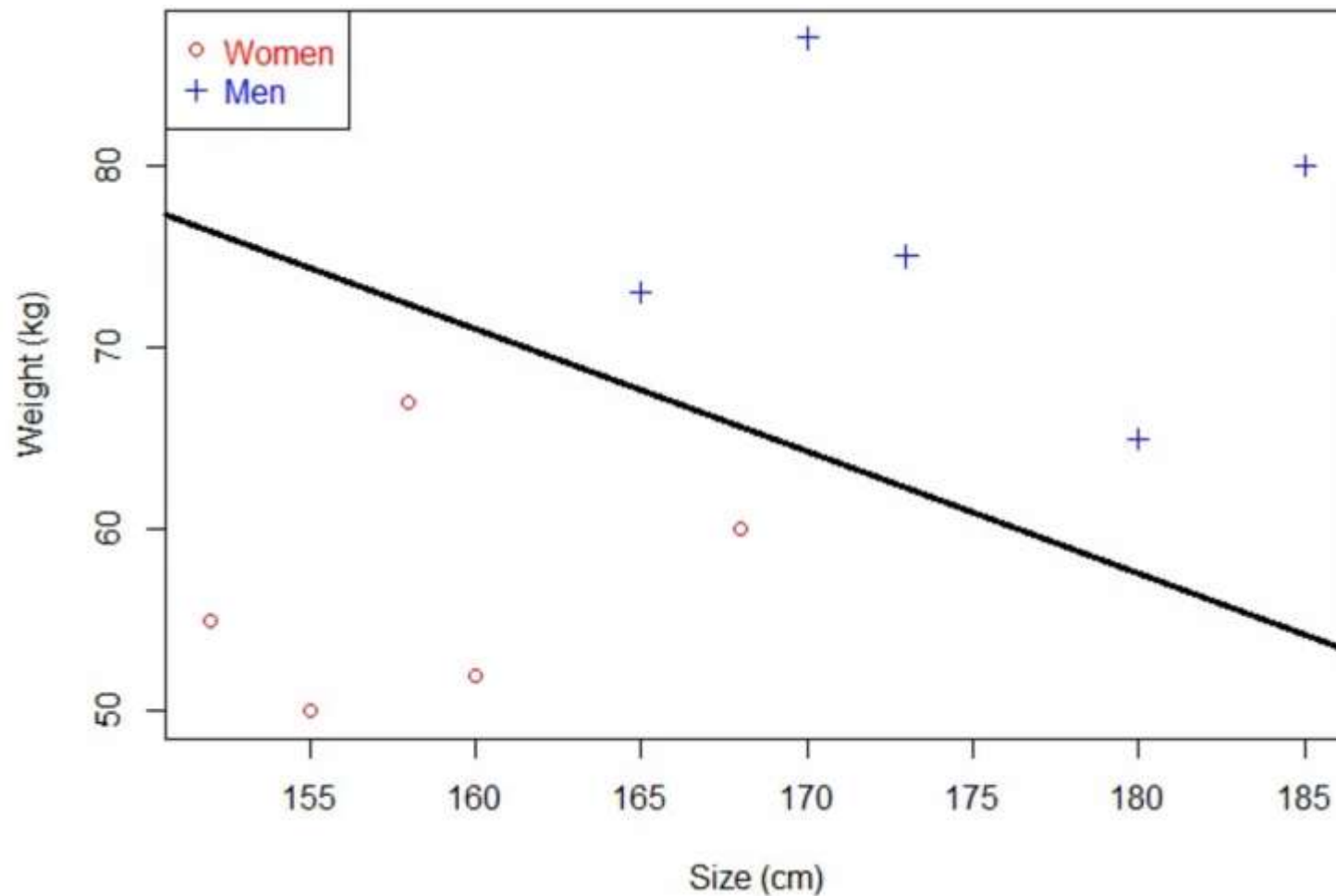
最优分离超平面

- 绿色的分离超平面在现实生活中的数据上进行分类，有一些女性被分错。
- 直观原因是这个分类超平面太接近女性（红色样本点），对noise/异常点/临近边界的点处理得并不好。



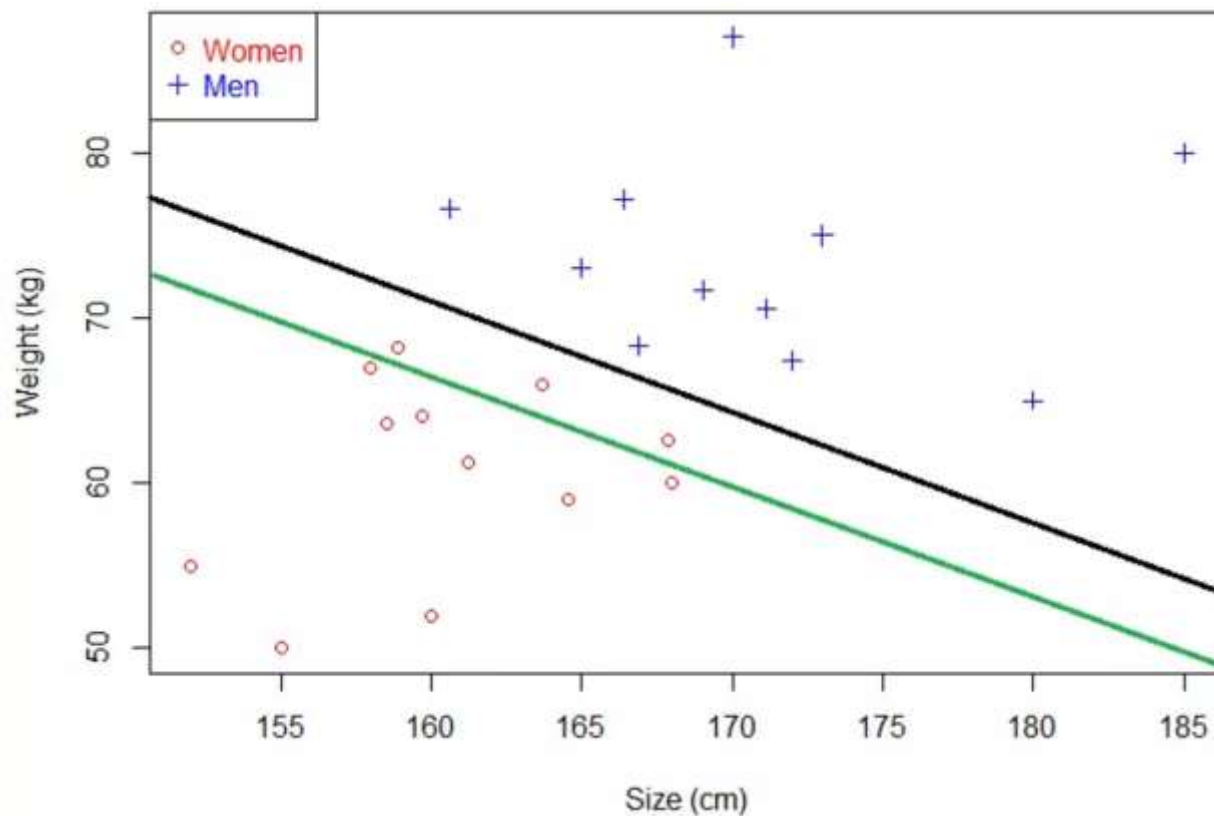
最优分离超平面

□ 要选择这
样一个超
平面 - 尽
可能的远
离所有类
别的数据
点：



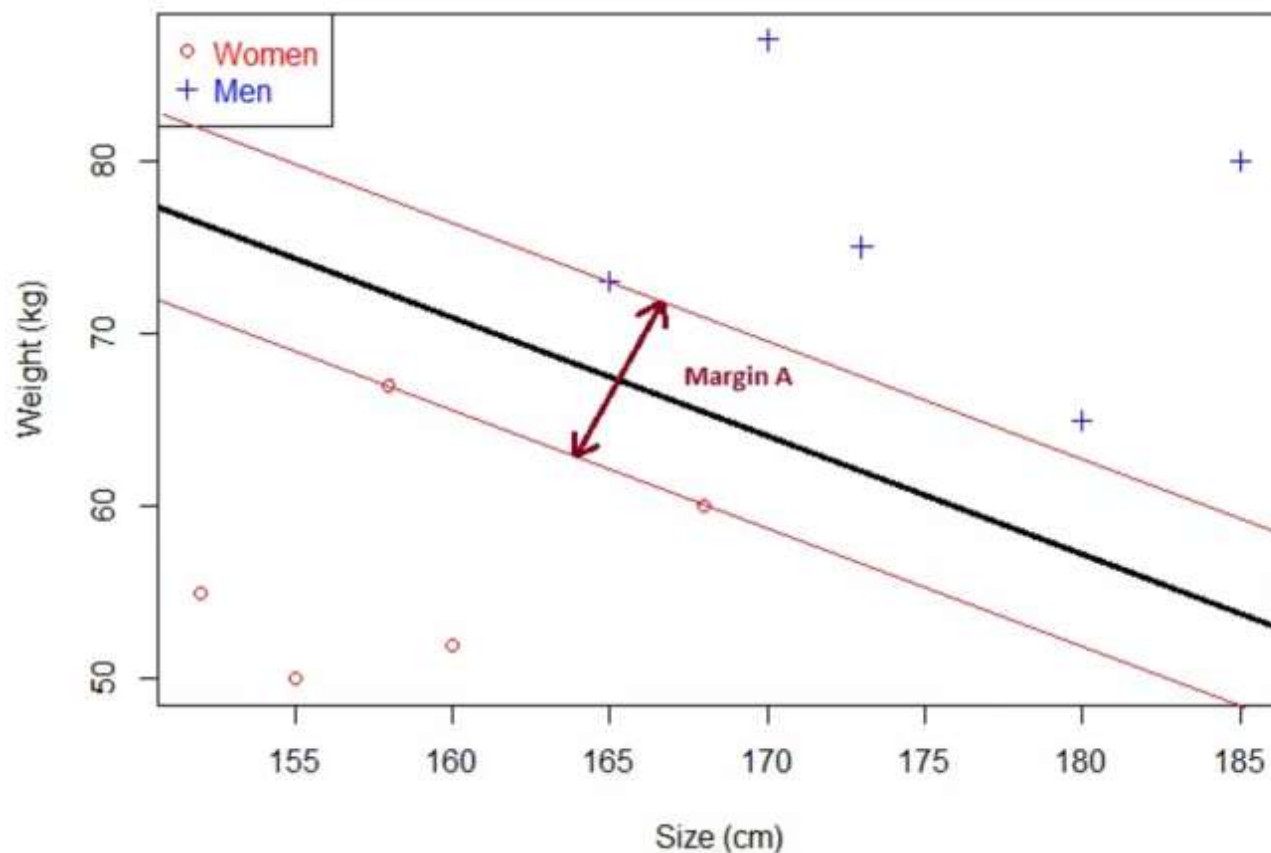
最优分离超平面

- 更加健壮
- 就是要找到这样一个最优分类超平面，保证：
 - ✓ 正确地对**训练数据**进行分类
 - ✓ 对**未知数据**也要进行很好的分类



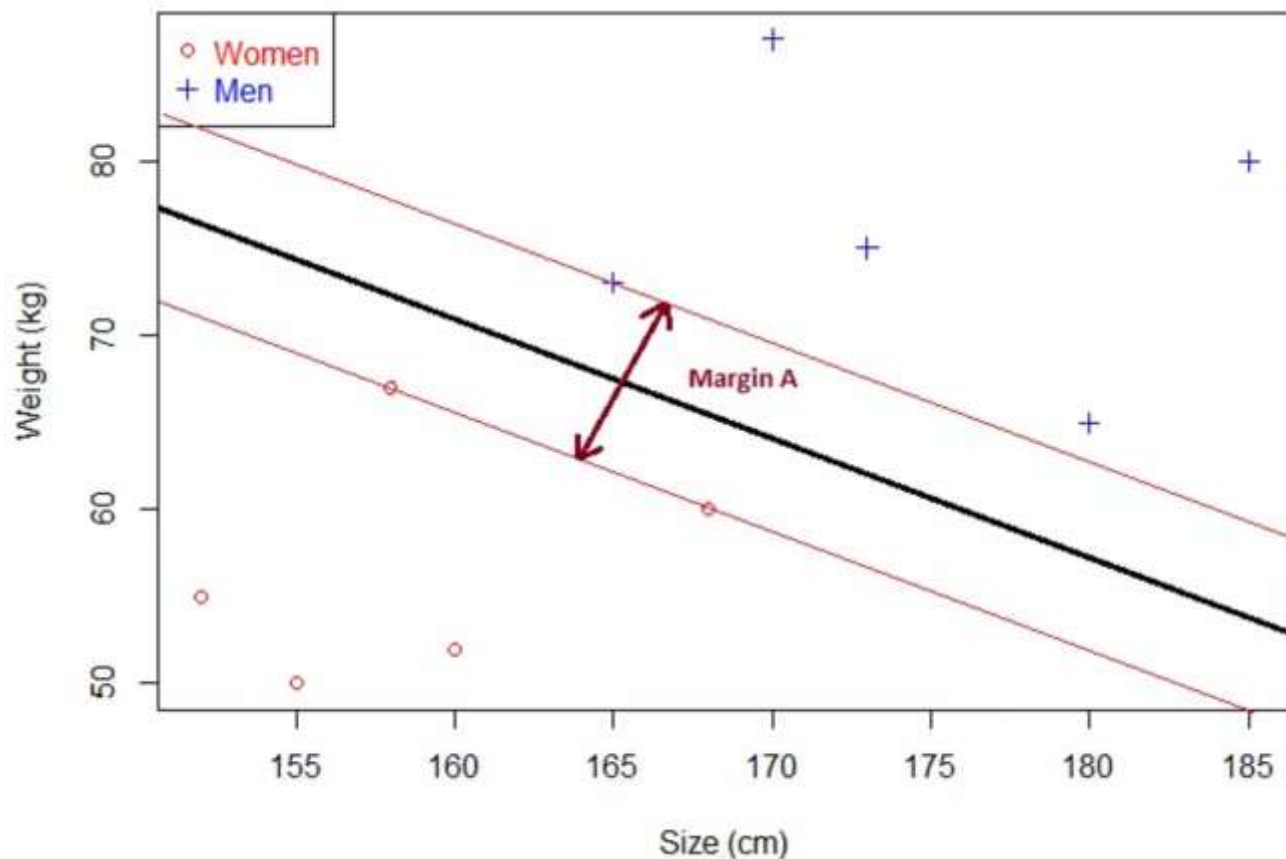
最优分离超平面的间隔

- Margin就是最优分离超平面的**间隔**。
- 给定一个特定的超平面，可以计算出这个超平面与和它最接近的数据点之间的距离。间隔（Margin）就是**二倍的这个距离**。



最优分离超平面的间隔

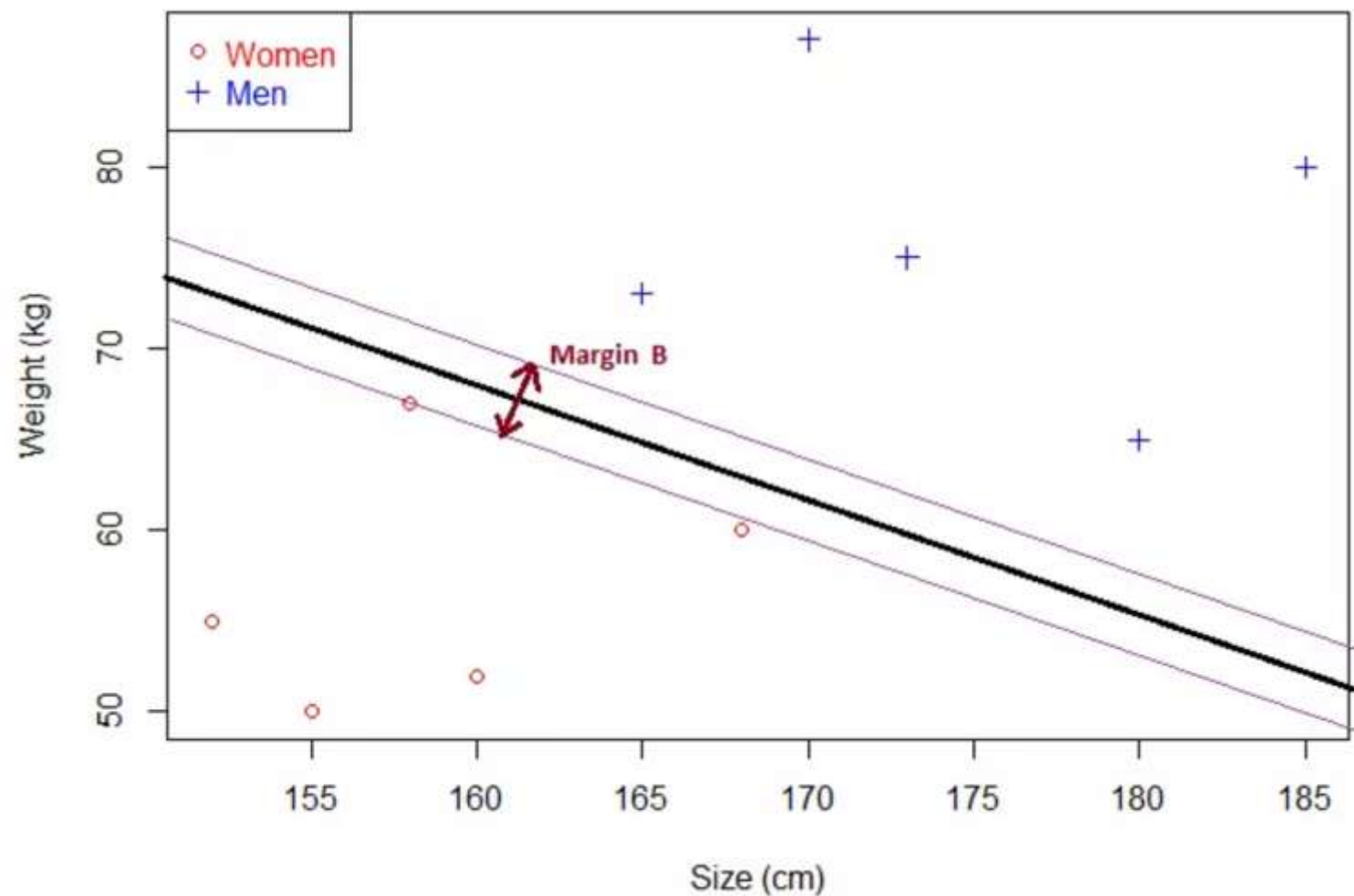
- 一般来说，间隔（Margin）中间是无点区域。这意味着里面不会有任何点。（注：在数据有噪声的情况下，可能达不到这么理想的状况，所以后面会引入**软间隔分类器**）



最优分离超平面的间隔

□ 从刚才的分离超平面里再选一个，间隔会看起来像这样：

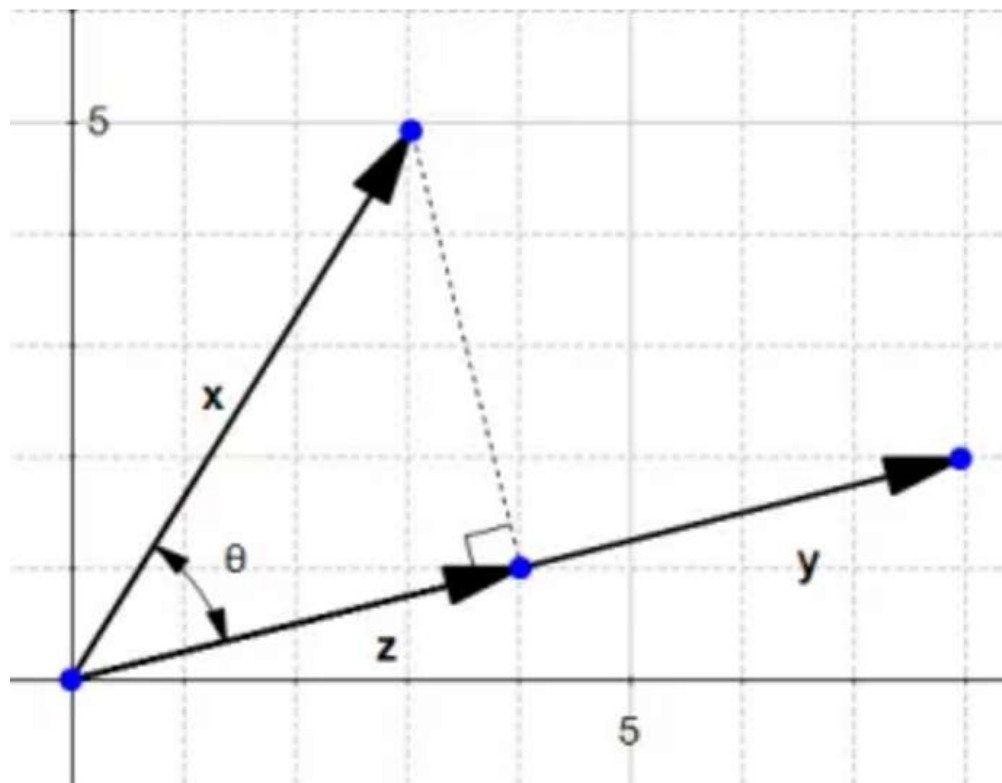
□ 可以清楚地看出，**B**的间隔要小于**A**的间隔。



最优分离超平面的间隔

□ 有以下的观察结果：

- 如果一个超平面有非常接近的一个数据点，那么间隔会小。
- 如果一个超平面离数据点越远，那么间隔越大。
- 这意味着**最优超平面将有最大的间隔**。
- **SVM是利用间隔最大化求得最优分离超平面。**
- **如何求距离呢？一个点（向量）到直线（超平面）的距离**



SVM的超平面

- 一条直线的数学方程是: $y = ax + b$ 等价于 $y - ax - b = 0$
- 超平面会被定义成类似的形式: $\mathbf{w}^T \mathbf{x} = 0$ (适用于高维空间, 向量 $\mathbf{w}$ 与超平面垂直 (方便计算点到超平面的距离))
- 有两个向量 $\mathbf{w}$ 和 $\mathbf{x}$:

$$\mathbf{w} \begin{pmatrix} -b \\ -a \\ 1 \end{pmatrix} \text{ 与 } \mathbf{x} \begin{pmatrix} 1 \\ x \\ y \end{pmatrix} \quad \begin{aligned} \mathbf{w}^T \mathbf{x} &= -b \times (1) + (-a) \times x + 1 \times y \\ \mathbf{w}^T \mathbf{x} &= y - ax - b \end{aligned}$$

计算点到超平面的距离

□ 设有一个超平面将两组数据划分开。

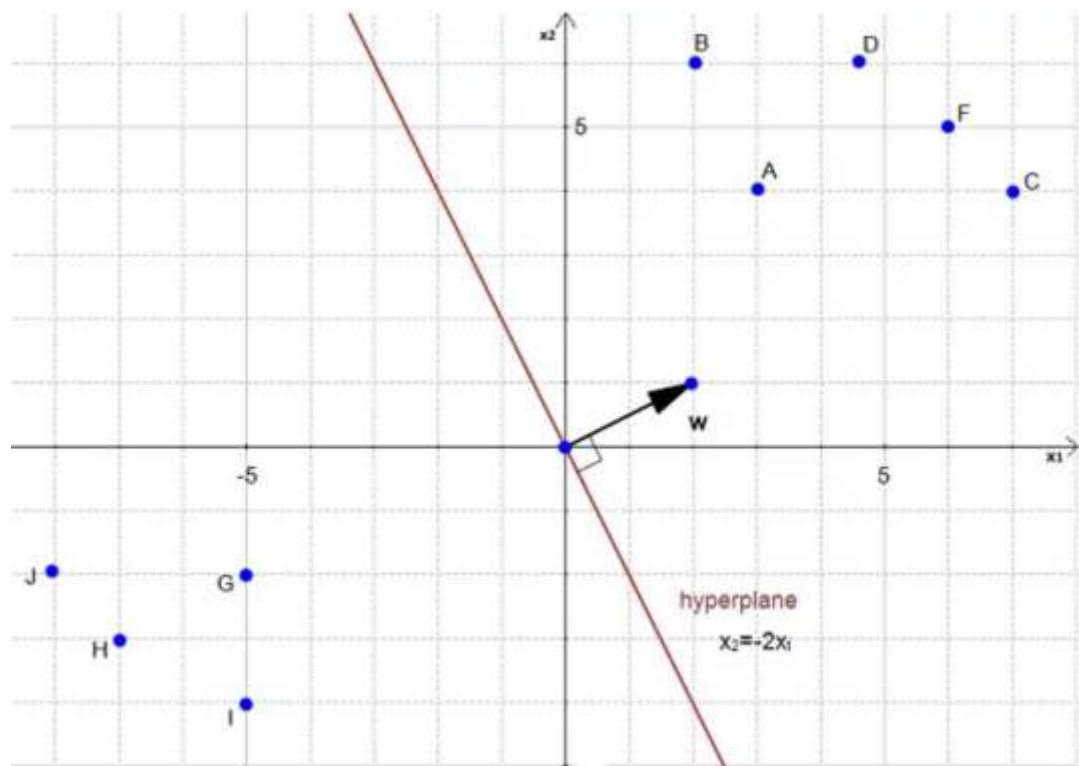
□ 简化起见将 w_0 设为0

□ 图中的超平面方程 $x_2 = -2x_1$

□ 相当于

$$\mathbf{w}^T \mathbf{x} = 0$$

$$\mathbf{w} \begin{pmatrix} 2 \\ 1 \end{pmatrix}, \mathbf{x} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$$

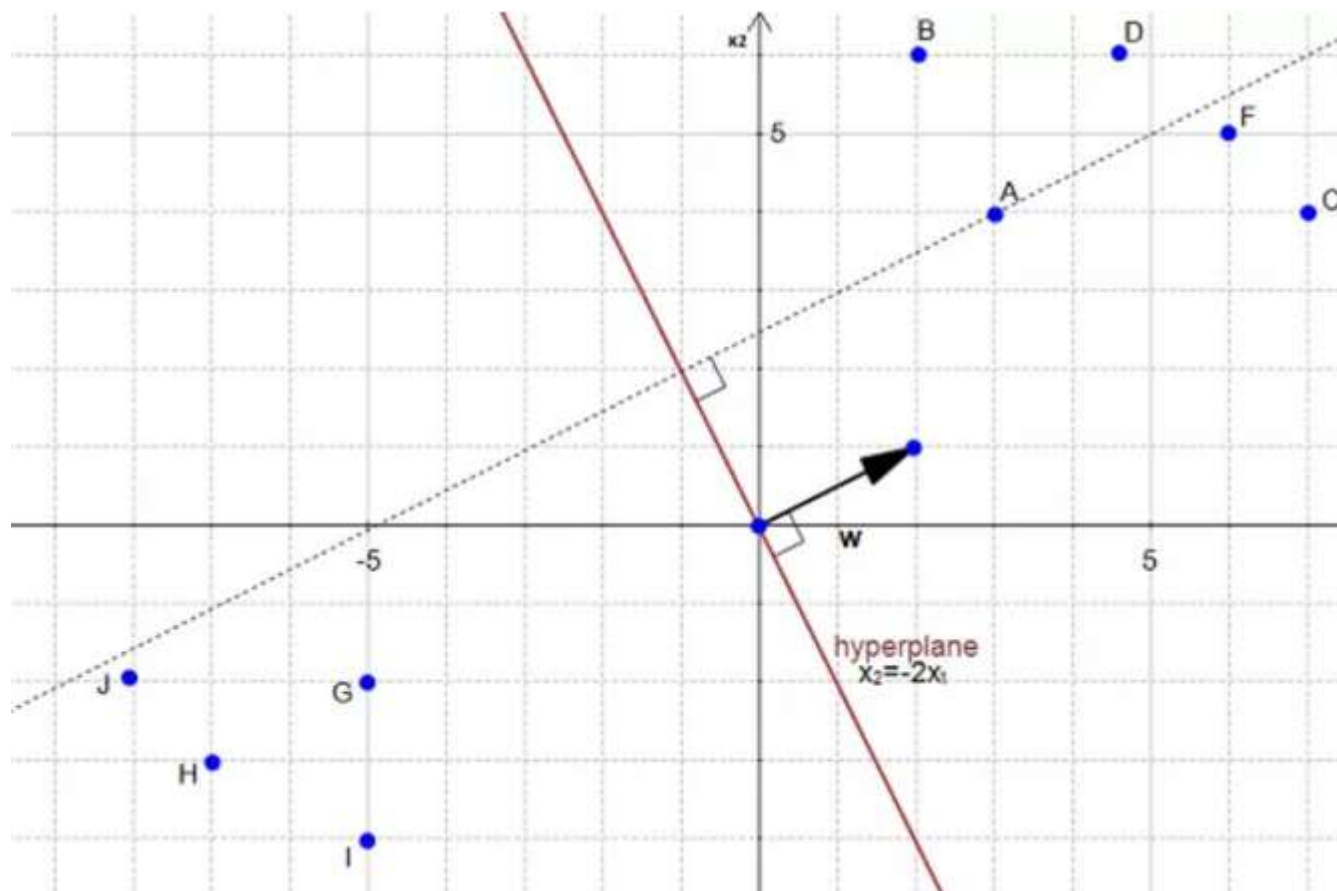


计算点到超平面的距离

□ 计算一下点

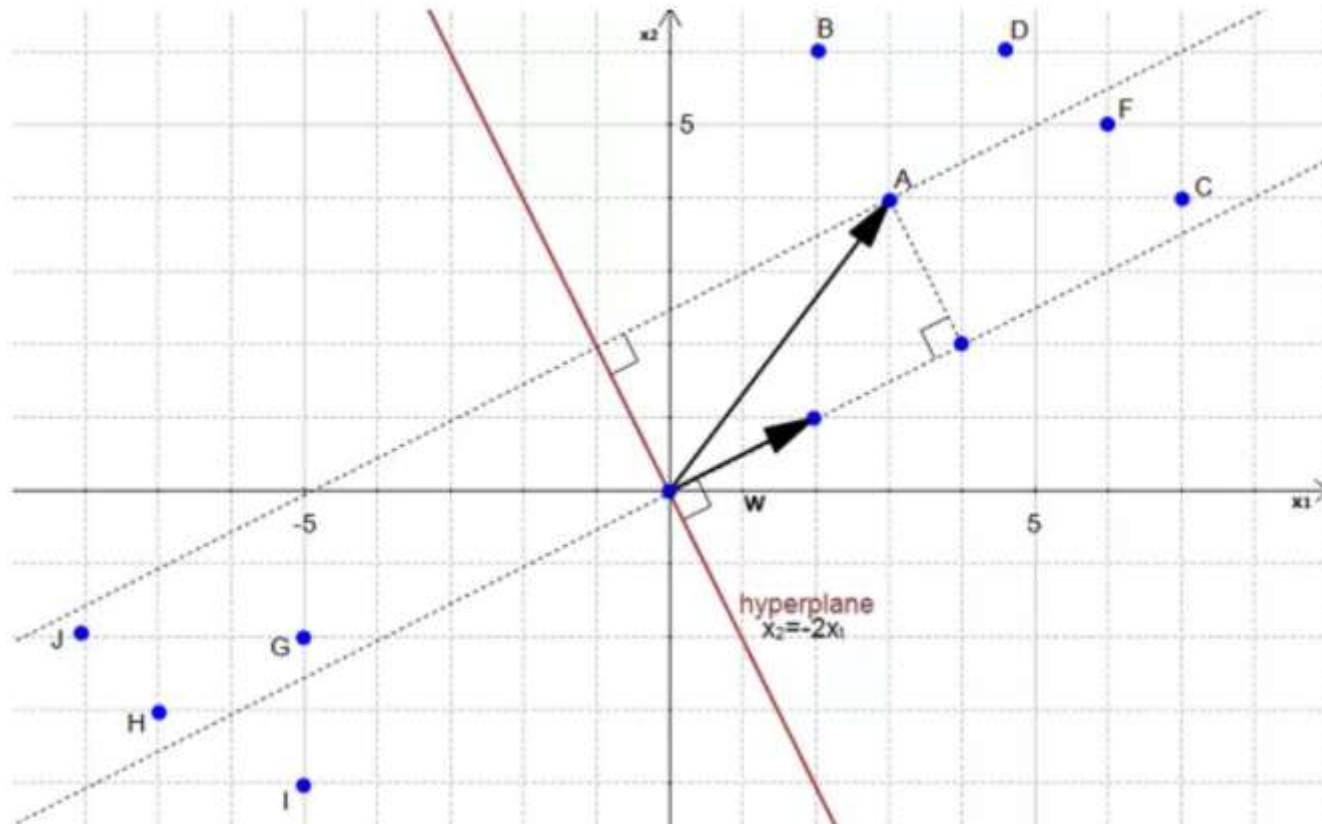
A (3, 4) 到超平面的
距离,

□ 下图是A投影到
超平面的距离。



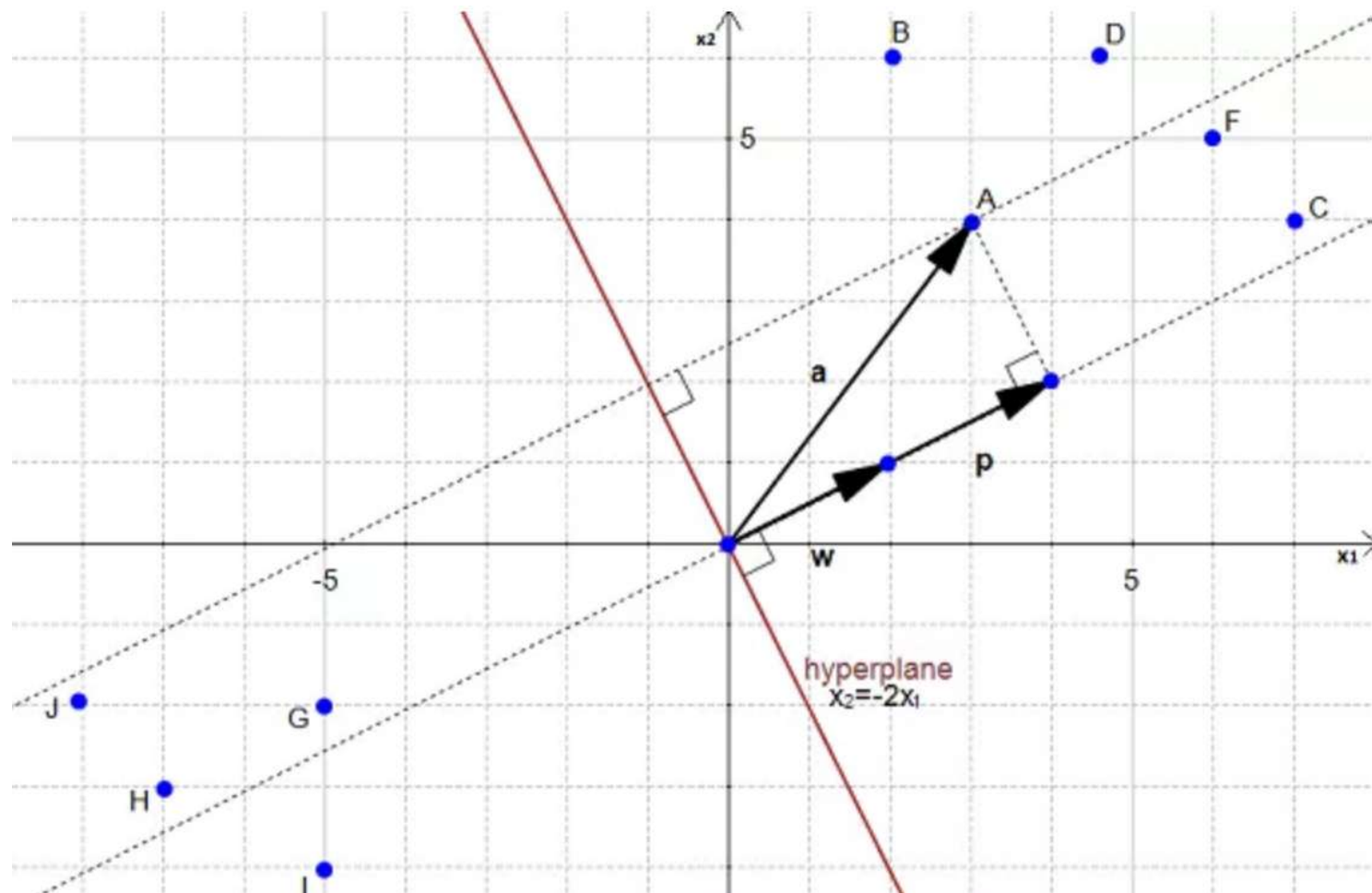
计算点到超平面的距离

- 把点A视作一个从原点指向A的向量。再将A向量投影到向量w上。



计算点到超平面的距离

□ 得到向量p



计算点到超平面的距离

□ 目标是计算点A(3, 4)和超平面之间的距离。从图中可以很清楚看到这个距离就是 $\|p\|$ 。向量 $a=(3, 4)$ 。

□ 向量 $w=(2, 1)$ 垂直于超平面 $\|w\| = \sqrt{2^2 + 1^2} = \sqrt{5}$

□ 设向量 u 为 w 的方向向量 $u = (\frac{2}{\sqrt{5}}, \frac{1}{\sqrt{5}})$

□ p 是 a 在 w 上的投影，所以：

$$p = (u \cdot a)u \quad p = (4, 2)$$

$$p = (3 \times \frac{2}{\sqrt{5}} + 4 \times \frac{1}{\sqrt{5}})u$$

$$p = (\frac{6}{\sqrt{5}} + \frac{4}{\sqrt{5}})u = \frac{10}{\sqrt{5}}u$$

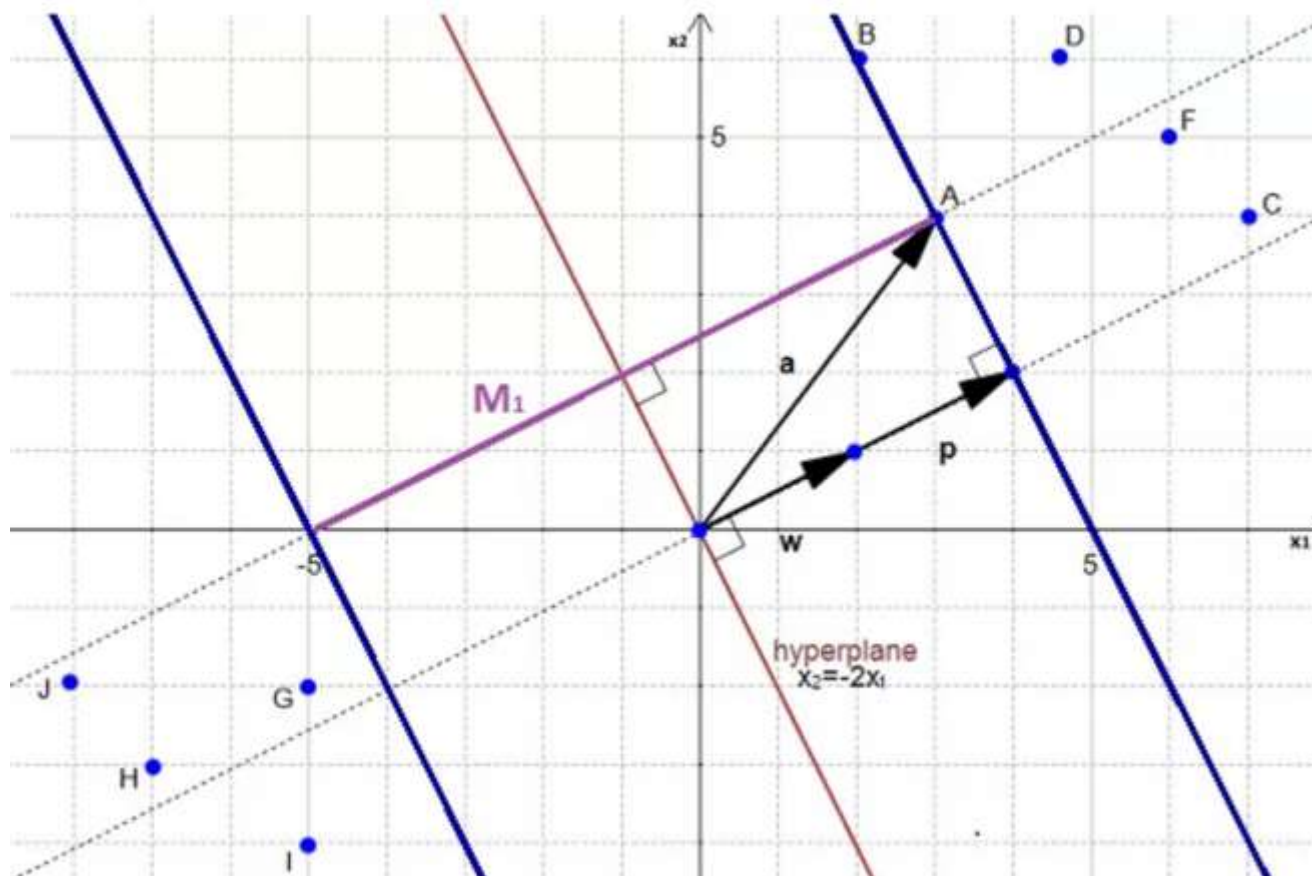
$$p = (\frac{10}{\sqrt{5}} \times \frac{2}{\sqrt{5}}, \frac{10}{\sqrt{5}} \times \frac{1}{\sqrt{5}})$$

$$p = (\frac{20}{5}, \frac{10}{5})$$

$$\|p\| = \sqrt{4^2 + 2^2} = 2\sqrt{5} \quad \text{margin} = 2\|p\| = 4\sqrt{5}$$

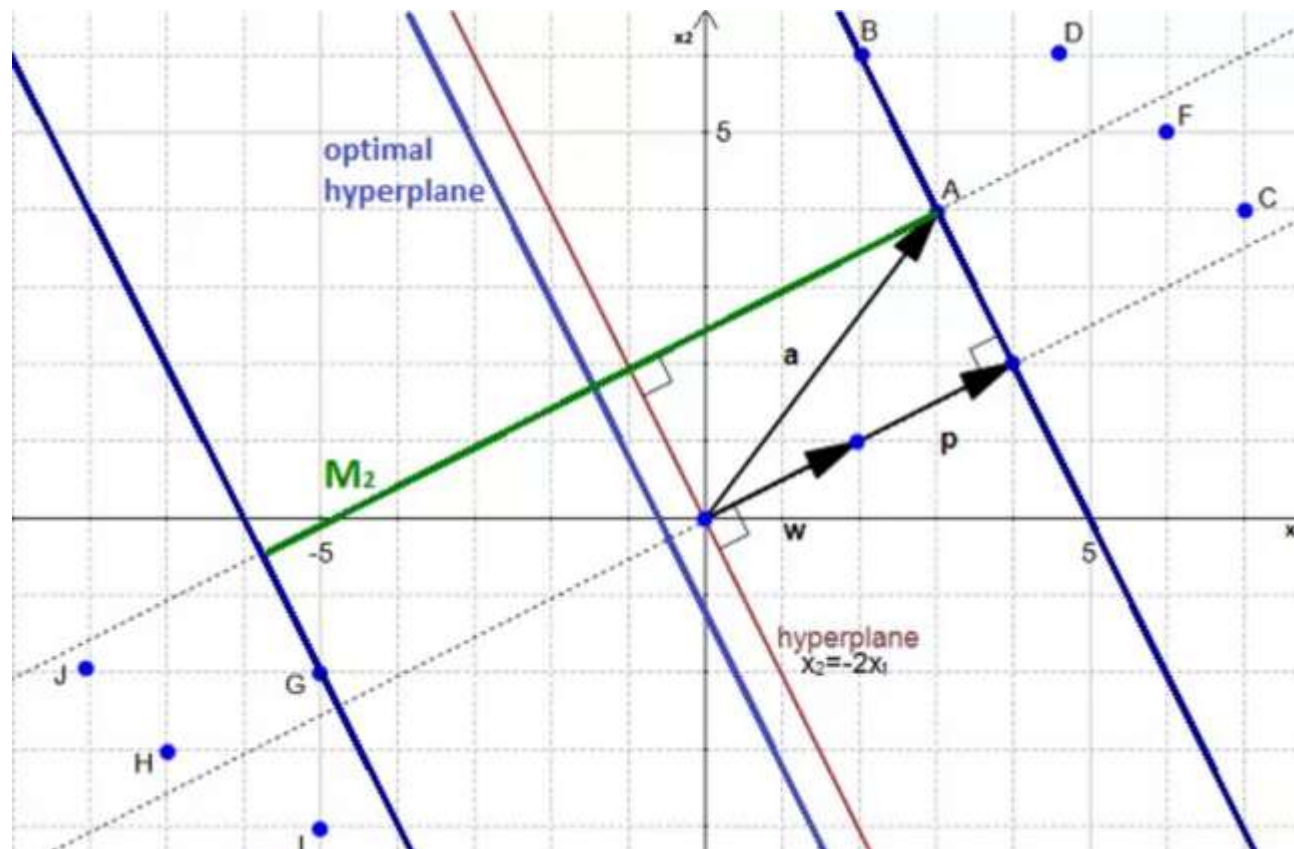
如何找到最优超平面

- 刚才的分类面不是最优
- 原因是未过G点



如何找到最优超平面

- 最优超平面，在刚才找的最初的超平面的稍左位置。最大间隔为 M_2 。
- 寻找最大间隔，就是寻找最优超平面



如何找到最大间隔

- 读取数据集。
- 找到两个平行超平面，可以划分数据并且两平面之间没有数据点。
- 最大化上述两个超平面间隔

如何找到最大间隔

□ 给出一个超平面 H_0 划分数数据集并且满足: $\mathbf{w} \cdot \mathbf{x} - b = 0$

□ 选择其他两个划分数数据集的超平面的 H_1 和 H_2 , 有如下的方程:

$$\mathbf{w} \cdot \mathbf{x} - b = \delta \quad \mathbf{w} \cdot \mathbf{x} - b = -\delta$$

□ 因为 $\mathbf{w}$ 和 b 是可以缩放的, 这里的变量 δ 其实是没有必要的, 为了简化(不引入那么多未知数), 通过一定的缩放, 让 $\delta=1$ 来简化问题。

$$\begin{aligned}\mathbf{w} \cdot \mathbf{x} - b &= 1 \\ \mathbf{w} \cdot \mathbf{x} - b &= -1\end{aligned}$$

如何找到最大间隔

- 还要确保 H_1 和 H_2 它们之间没有任何的数据点。只选择符合以下两个约束的超平面

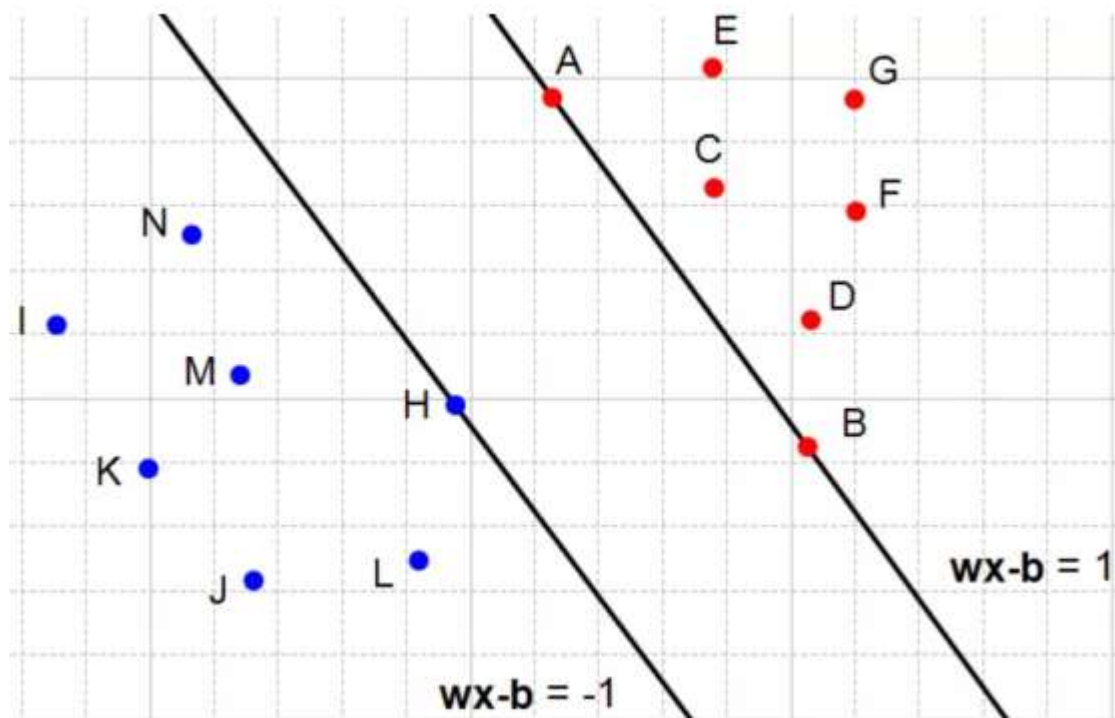
$$\mathbf{w} \cdot \mathbf{x}_i - b \geq 1 \text{ for } \mathbf{x}_i \text{ 属于类别 } 1$$

$$\mathbf{w} \cdot \mathbf{x}_i - b \leq -1 \text{ for } \mathbf{x}_i \text{ 属于类别 } -1$$

- 统一起来，等价于： $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$ for all $1 \leq i \leq n$

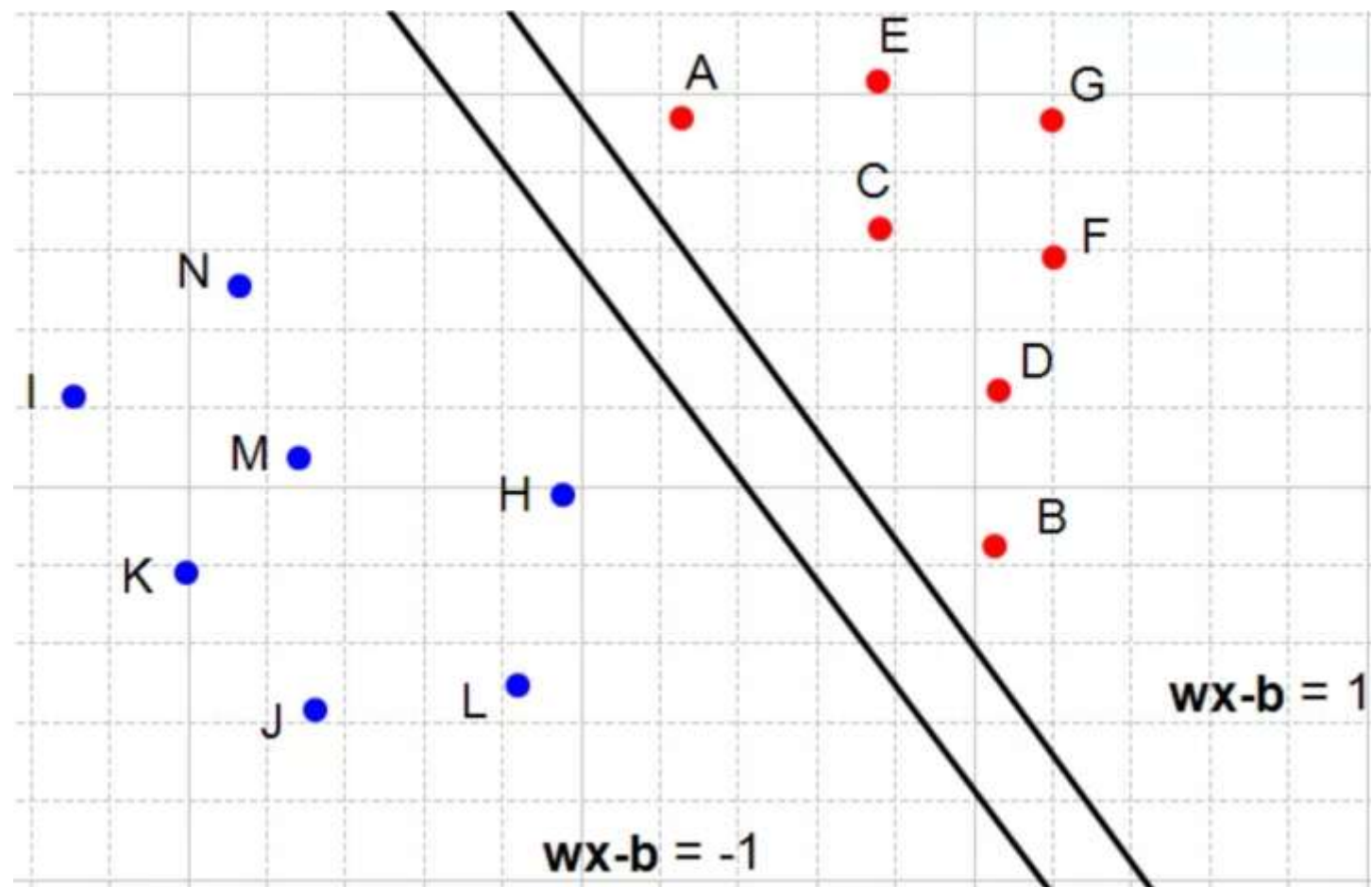
如何找到最大间隔

□ 所有的红点都是1类，所有的蓝点都是-1类。需要验证它不违反约束 $w \cdot x_i - b \geq 1$ 。当 $x_i = A$ 代入超平面方程，发现满足 $w \cdot x_i - b = 1$ ，同样的B点也是。当 $x_i = C$ 超平面上的点满足 $w \cdot x_i - b > 1$ ，同样的D、E、F、G点也是。类似的，对于-1类的点也满足约束。



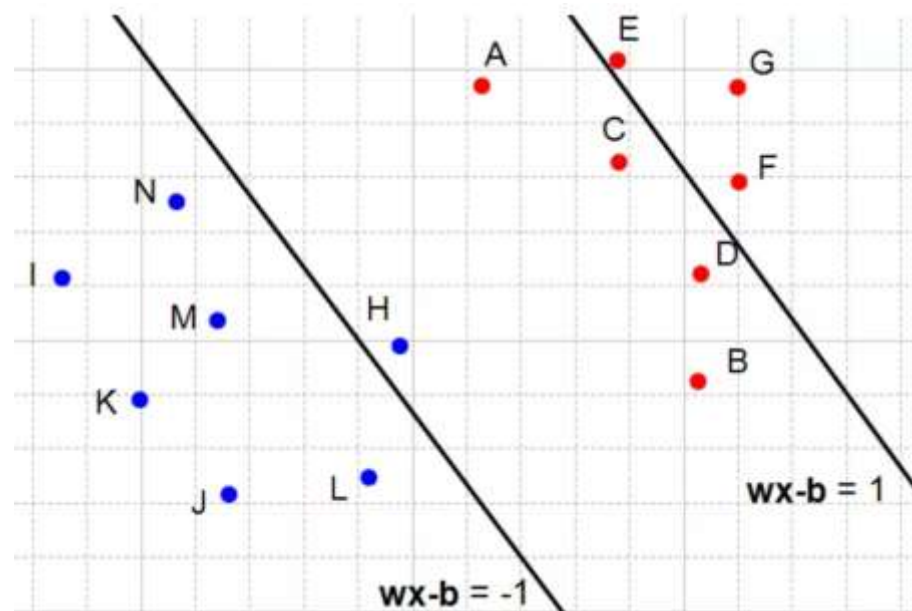
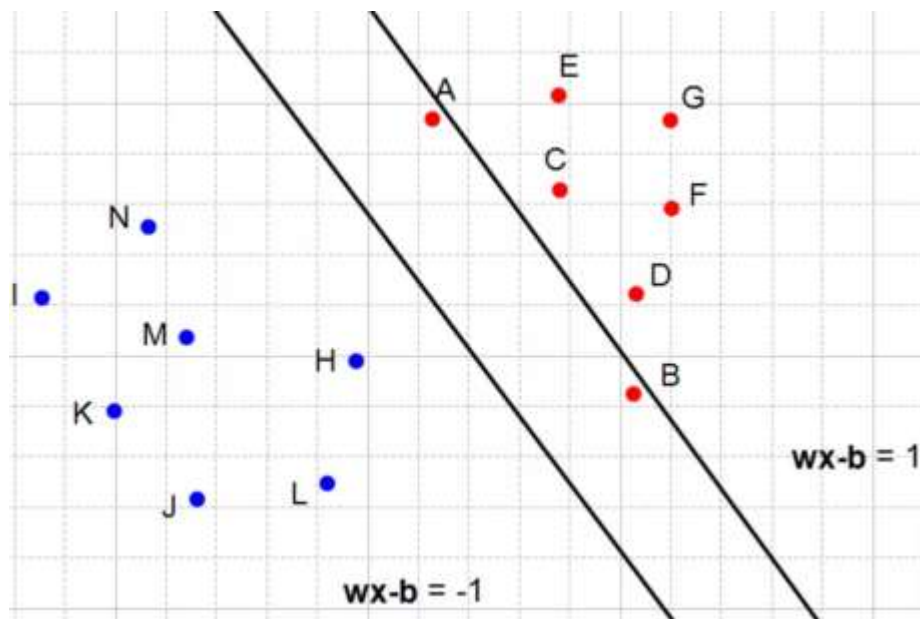
如何找到最大间隔

□ 满足约束
条件的一
对超平面



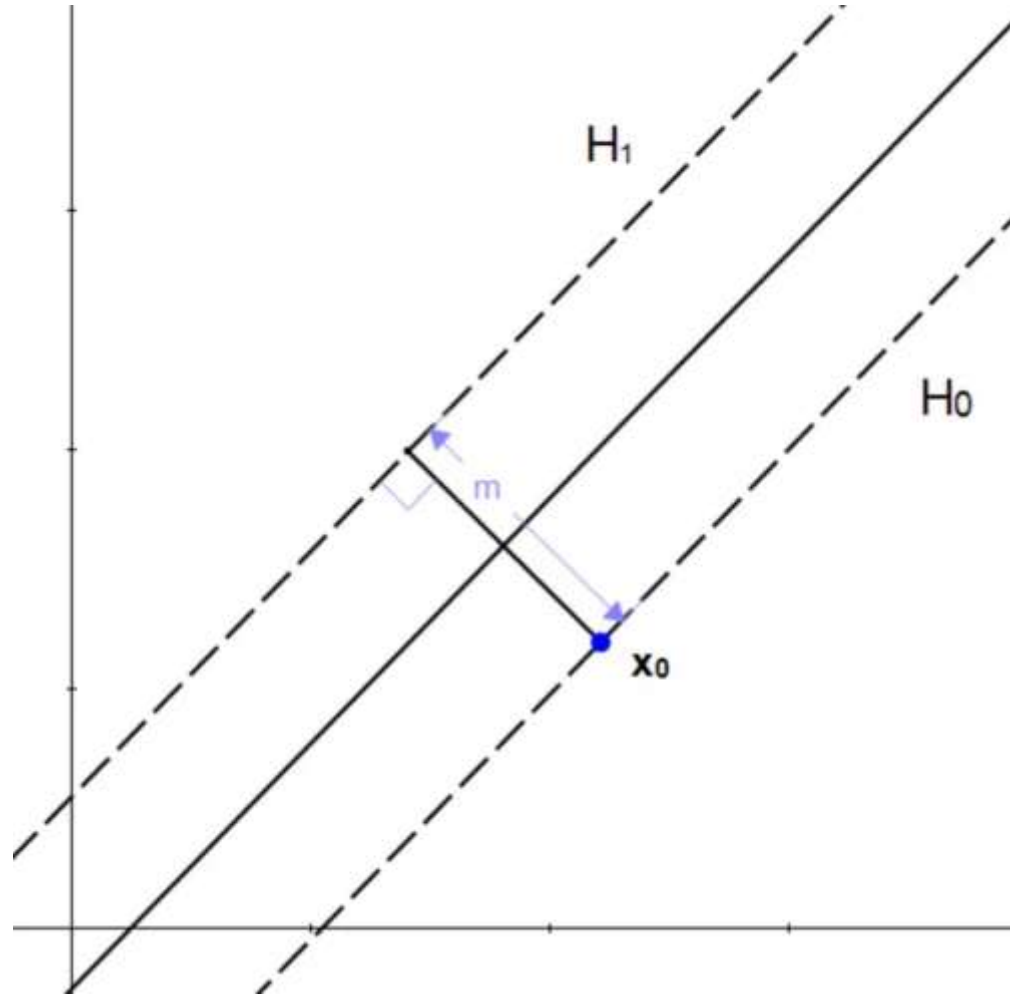
如何找到最大间隔

□ 不满足约束条件的一对超平面：超平面之间有了数据点



超平面之间距离的计算

- 已知 H_0 是满足约束 $w \cdot x - b = -1$ 的超平面， H_1 是满足约束 $w \cdot x - b = 1$ 的超平面
- x_0 是 H_0 上的一点
- 从 x_0 到超平面 H_1 的垂直距离 m 就是间隔。 x_0 在超平面 H_0 上，所以 m 是超平面 H_0 与 H_1 间的距离。
- 现在试着找出 m 的值。



超平面之间距离的计算

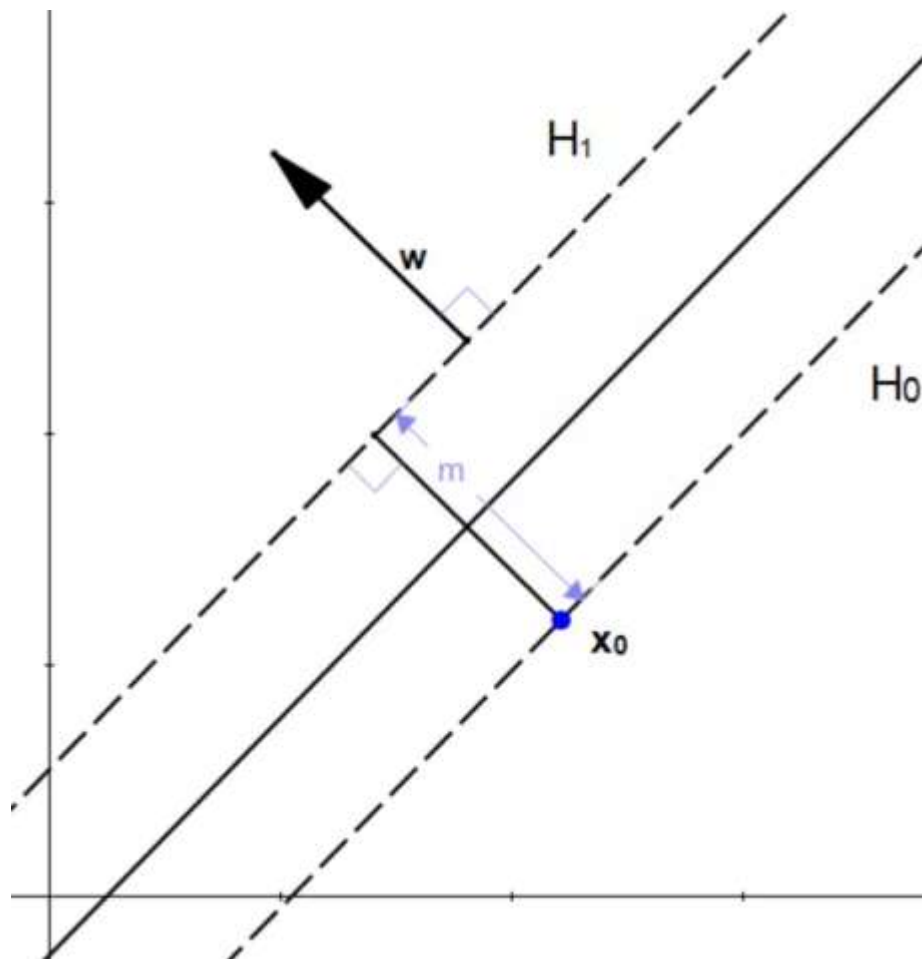
- 如果 x_0 和另一个超平面距离为 m ，加上 m 应该会得到另一个超平面上的点。
- 不过 m 是一个标量， x_0 是一个向量，无法运算。需要将 m 变换成一个向量。

这个向量满足：

- 长度为 m
- 方向垂直于平面 H_1
- 仔细看看超平面的形式，发现已经有一个垂直 H_1 的向量，正是 w （因为 H_1 为 $w \cdot x - b = 1$ ）

超平面之间距离的计算

- 但是长度不为 m 。
- 怎么办？

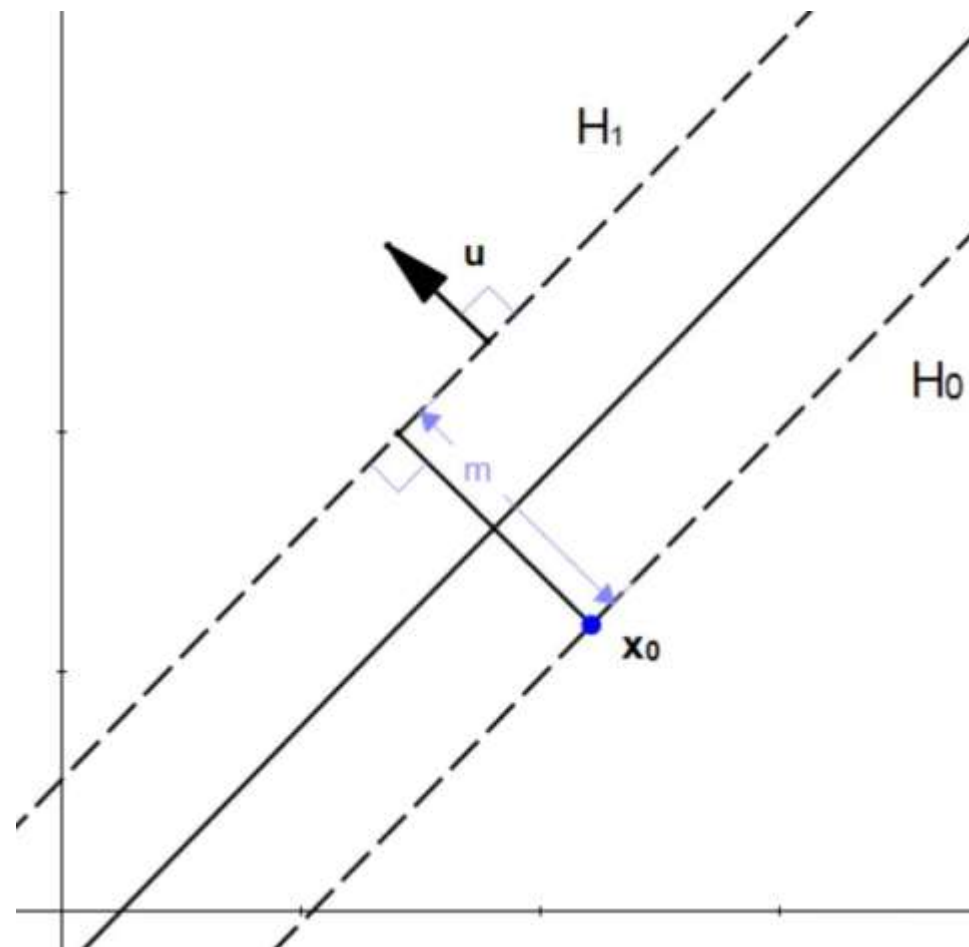


超平面之间距离的计算

- 定义 u 是 w 的单位向量即 $\|u\| = 1$ 且它和 w 有相同的方向，垂直于平面 H_1 。

$$u = \frac{w}{\|w\|}$$

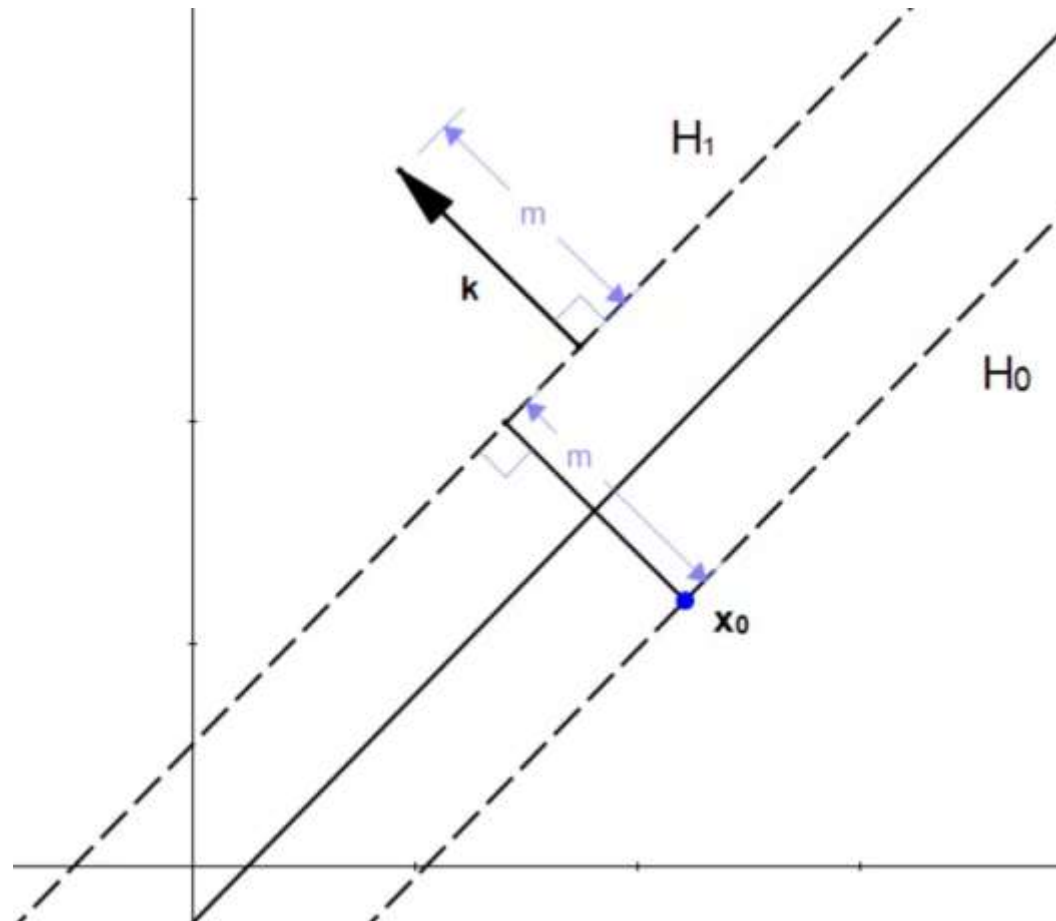
- 如果用 u 乘以 m ，得到向量 $k=mu$ ，则 k 就是正在寻找的向量。



超平面之间距离的计算

- 这样把 m 转换成一个可以和向量 x_0 相加的向量 k 。

$$\mathbf{k} = m\mathbf{u} = m \frac{\mathbf{w}}{\|\mathbf{w}\|}$$



超平面之间距离的计算

- 在向量 x_0 上加上 k ，可以得到向量 $z_0=x_0+k$ （终点在超平面 H_1 上）

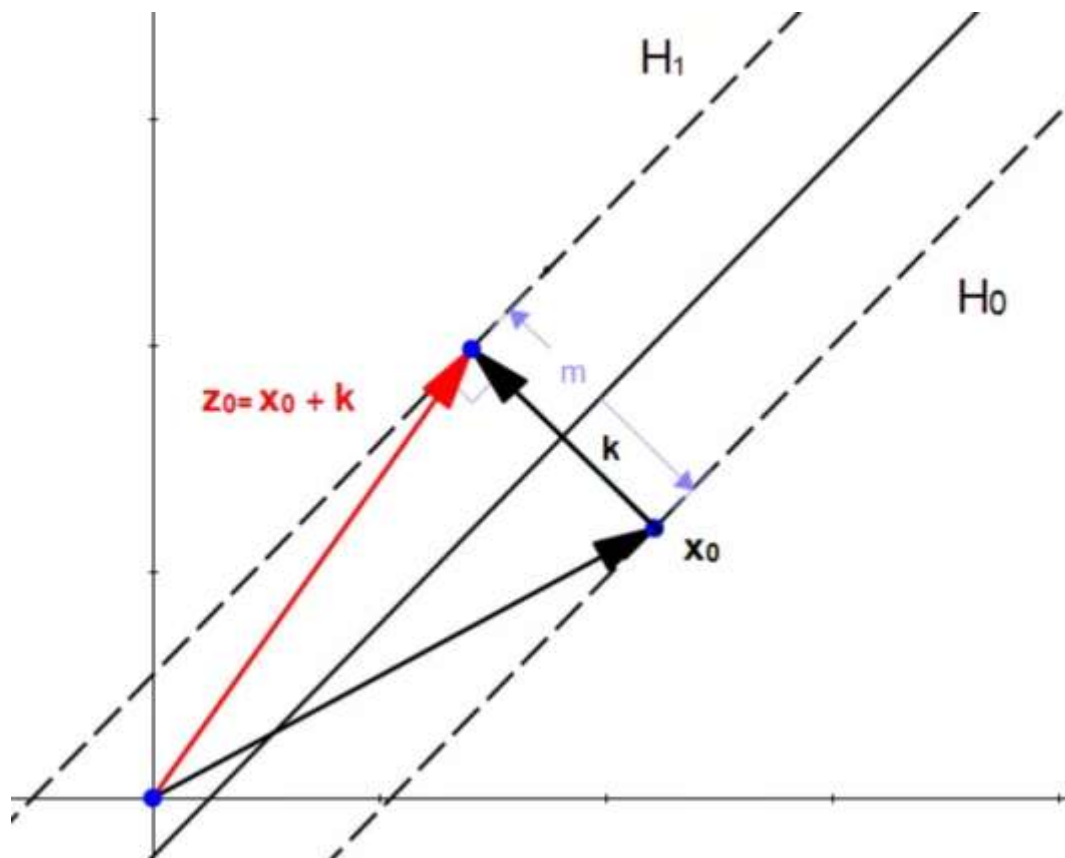
$$\mathbf{w} \cdot \mathbf{z}_0 + b = 1$$

$$\mathbf{w} \cdot (\mathbf{x}_0 + \mathbf{k}) + b = 1$$

$$\mathbf{w} \cdot \left(\mathbf{x}_0 + m \frac{\mathbf{w}}{\|\mathbf{w}\|} \right) + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m \frac{\mathbf{w} \cdot \mathbf{w}}{\|\mathbf{w}\|} + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + m \frac{\|\mathbf{w}\|^2}{\|\mathbf{w}\|} + b = 1$$



超平面之间距离的计算

□ 因为 $\mathbf{x}_0$ 在 H_0 上, 所以 $\mathbf{w} \cdot \mathbf{x}_0 + b = -1$, 代入:

$$\mathbf{w} \cdot \mathbf{x}_0 + m\|\mathbf{w}\| + b = 1$$

$$\mathbf{w} \cdot \mathbf{x}_0 + b = 1 - m\|\mathbf{w}\|$$

$$-1 = 1 - m\|\mathbf{w}\|$$

$$m\|\mathbf{w}\| = 2$$

$$m = \frac{2}{\|\mathbf{w}\|}$$

□ 找到了一种非常简便的计算 m 的方法

最大化超平面之间距离

□ 计算间隔 $m = \frac{2}{\|\mathbf{w}\|}$

□ 很容易发现， w 的模越大，间隔越小。最大化间隔也就是最小化 w 的模。

□ 在所有满足约束条件超平面中，选择有最小 $\|\mathbf{w}\|$ 的超平面，因为这意味着有最大间隔。有了以下的有约束最小化优化问题：

约束条件为 $y_i(\mathbf{w} \cdot \mathbf{x}_i - b) \geq 1$ (对于 $i = 1, \dots, n$) 的情况下

找到 $\|\mathbf{w}\|$ 最小的 $\mathbf{w}, b$

□ 拉格朗日乘子法求解：典型的带约束条件的求极值问题，目标函数是 w 的二次函数，约束函数是 w 的线性函数：二次规划问题。求解二次规划问题的一般性方法就是添加拉格朗日乘子，构造拉格朗日函数

优化问题

- **无约束**：直接求目标函数偏导得0的点。如果没有解析解的话，可以使用**梯度下降或牛顿方法**等迭代的手段来使x沿负梯度方向逐步逼近极小值点。

$$\min_x f(x) \quad \nabla_x f(x) = 0$$

- **等式约束**：可以直接应用拉格朗日乘子法去求取最优值
- **不等式约束**：可以转化为在满足 KKT 约束条件下应用拉格朗日乘子法求解。
- 拉格朗日求得的并不一定是最优解，只有在凸优化的情况下，才能保证得到的是最优解。

拉格朗日乘子法

- 求解最小化问题，一个直观的想法是如果能够构造一个函数，使得该函数在可行解区域内与原目标函数完全一致，而在可行解区域外的数值非常大，甚至是无穷大，那么这个没有约束条件的新目标函数的优化问题就与原来有约束条件的原始目标函数的优化问题是等价的问题。
- 使用拉格朗日方程的目的：约束条件放到目标函数中，从而将有约束优化问题转换为无约束优化问题。

拉格朗日乘子法

经典拉格朗日乘子是下面的优化问题：

$$\min_{\boldsymbol{x}} f(\boldsymbol{x}), \text{ s.t. } h(\boldsymbol{x}) = 0$$

引入拉格朗日函数：

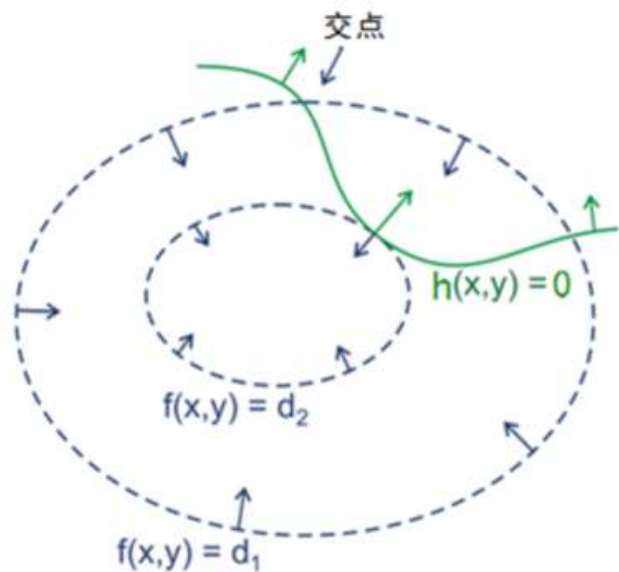
$$\mathcal{L}(\boldsymbol{x}, \lambda) = f(\boldsymbol{x}) - \lambda h(\boldsymbol{x})$$

求解该函数可得：

$$\begin{cases} \nabla_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}, \lambda) = \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) - \lambda \nabla_{\boldsymbol{x}} h(\boldsymbol{x}) = 0 \\ \nabla_{\lambda} \mathcal{L}(\boldsymbol{x}, \lambda) = -g(\boldsymbol{x}) = 0 \end{cases} \rightarrow \begin{cases} \nabla_{\boldsymbol{x}} f(\boldsymbol{x}) = \lambda \nabla_{\boldsymbol{x}} h(\boldsymbol{x}) \\ g(\boldsymbol{x}) = 0 \end{cases}$$

几何直观：梯度共线

以目标函数 $f(x,y)$ 为例，其等高线为图中虚线所示；约束等式 $h(x,y)=0$ ，如图绿线所示。 $f(x,y)$ 和 $h(x,y)=0$ 只有三种可能：相交、相切和没有交集（无共同解）。而**最优点必须落相切点**上，因为如果两者相交而不相切，沿着约束曲面移动一小步，就能使目标函数值下降（或上升），意味着该点不是极值点；相切意味着在约束曲面的切方向上，目标函数的变化率为零。



相切在数学上表现为：

$$\nabla_x f(\mathbf{x}) = \lambda \nabla_x h(\mathbf{x})$$

目标函数的梯度和约束等式的法向量平行（共线），此时即得最优解。

等式约束优化问题

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & h_i(x) = 0, i = 1, 2, \dots, m \end{aligned}$$

推广到多个约束等式时，构建拉格朗日函数如下，其中拉格朗日乘子 $\alpha \in \mathbb{R}^m$

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = f(\mathbf{x}) - \sum_{i=1}^m \alpha_i h_i(\mathbf{x})$$

求解如下：

$$\begin{cases} \nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = 0 \\ \nabla_{\boldsymbol{\alpha}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}) = 0 \end{cases}$$

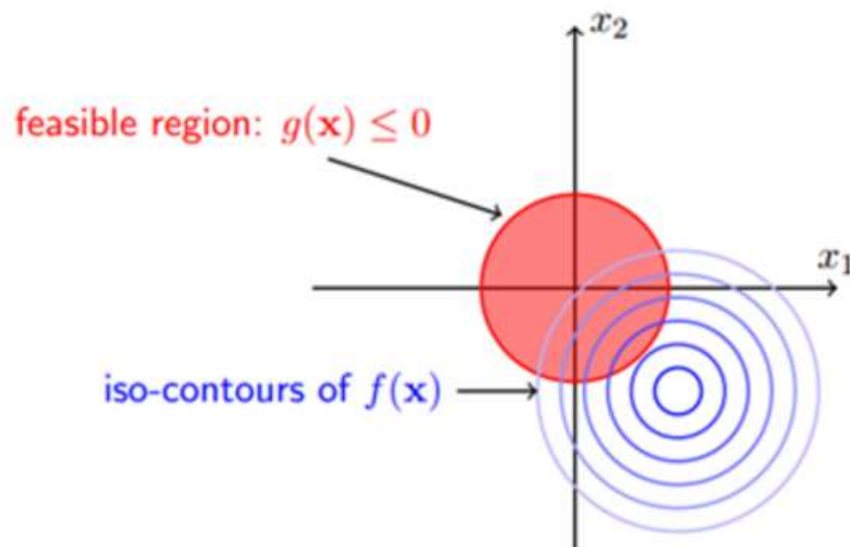
不等式约束优化问题

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad s.t. \quad g(\mathbf{x}) \leq 0$$

$g(\mathbf{x}) \leq 0$ 定义了一个可行域（包括边界和内部），对应的拉格朗日函数：

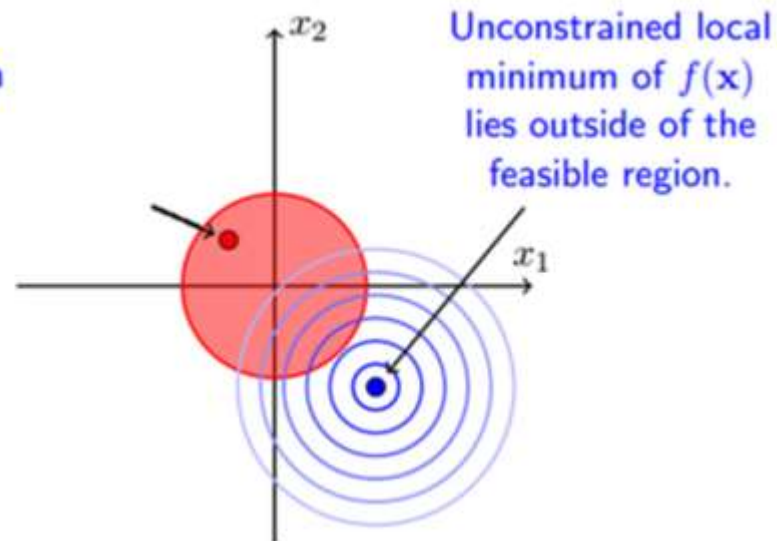
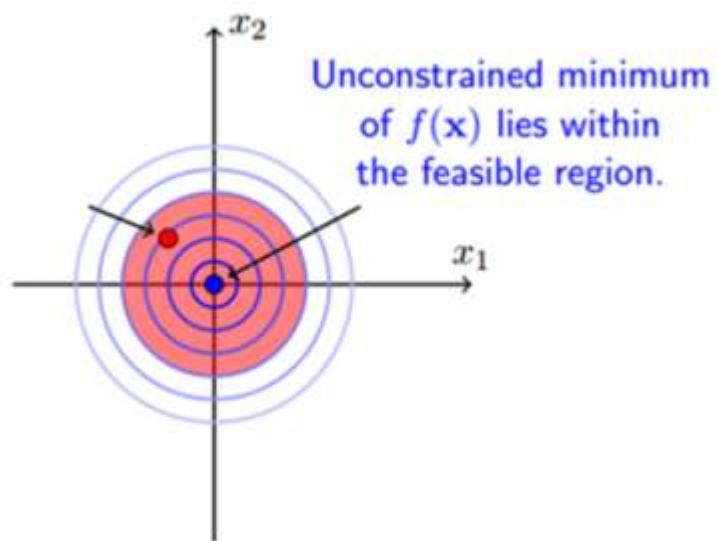
$$\mathcal{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$$

此时，可行解必须落在可行域之内，如图所示



不等式约束优化问题

- **约束不起作用** ($g(\mathbf{x}) < 0$) : 此时可行点位于可行域内部, 约束实际上不影响极值, 问题退化为无约束优化问题。最优点应满足 $\nabla_{\mathbf{x}} f(\mathbf{x}) = 0$, 此时 $\lambda = 0$ 。
- **约束起作用** ($g(\mathbf{x}) = 0$) : 此时最优点位于边界上, 类似于等式约束的情况。

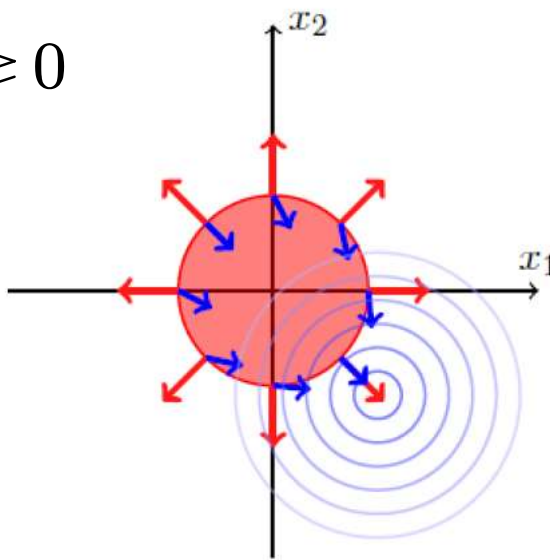


不等式约束优化问题

因此，可行解满足： $\lambda g(\mathbf{x}) = 0$

这里 $g(\mathbf{x}) = 0$ 时与等式约束有一个关键区别：不等式约束限制了可行方向只能指向约束曲面内侧（即 $g(\mathbf{x}) \leq 0$ 的区域）。在边界上，沿着约束曲面的切向移动可能进入内部或外部，但只有指向内部的方向是可行的（尽量靠近无约束的解），即约束函数的梯度方向与目标函数的负梯度方向相

同： $-\nabla_{\mathbf{x}} f(\mathbf{x}) = \lambda \nabla_{\mathbf{x}} g(\mathbf{x}), \quad \lambda \geq 0$



A local optimum occurs when $\nabla_{\mathbf{x}} f(\mathbf{x})$ and $\nabla_{\mathbf{x}} g(\mathbf{x})$ are parallel

不等式约束优化问题

推广到多个不等式或等式约束的一般化问题：

$$\begin{aligned} \min_{\mathbf{x}} f(\mathbf{x}) \\ s.t. \quad h_i(\mathbf{x}) = 0, \quad i = 1, 2, \dots, m \\ g_j(\mathbf{x}) \leq 0, \quad j = 1, 2, \dots, n \end{aligned}$$

构造广义拉格朗日函数：

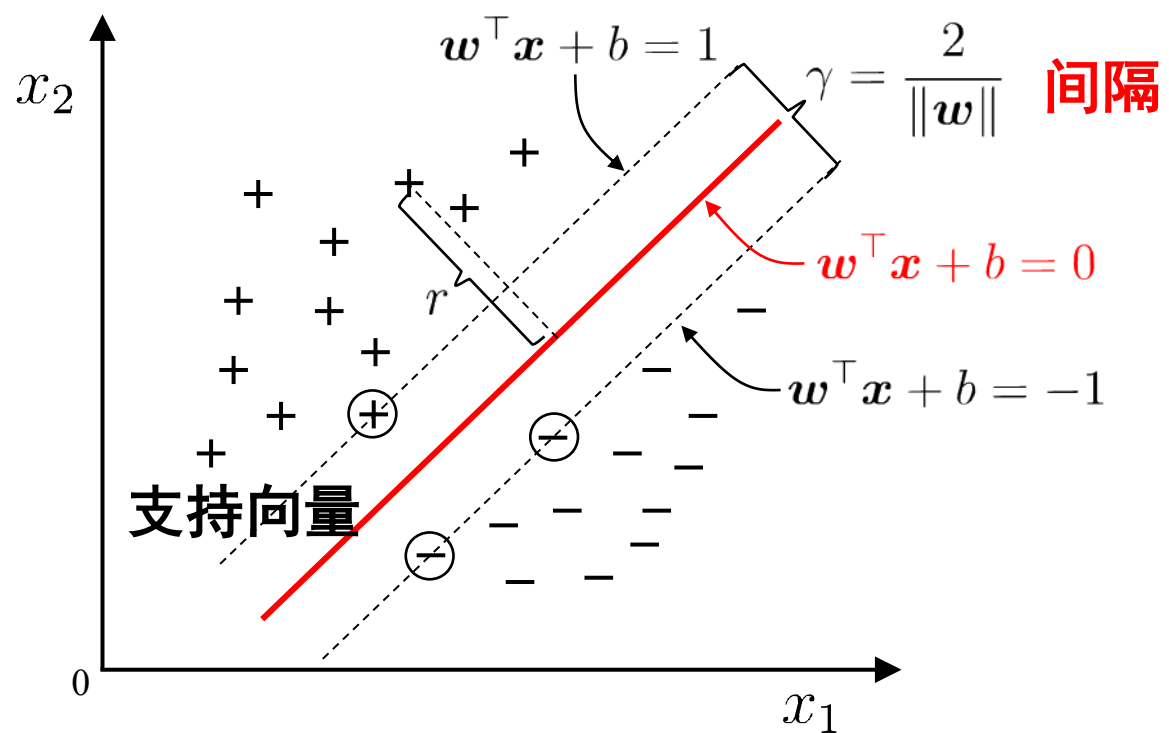
$$\mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = f(\mathbf{x}) + \sum_{i=1}^m \alpha_i h_i(\mathbf{x}) + \sum_{j=1}^n \beta_j g_j(\mathbf{x})$$

在最优解处（在一定正则性条件下），必须满足KKT（Karush-Kuhn-Tucker）条件：

- 梯度为零： $\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = 0$
- 原问题可行： $h_i(\mathbf{x}) = 0, \quad i = 1, 2, \dots, m$
 $g_j(\mathbf{x}) \leq 0, \quad j = 1, 2, \dots, n$
- 对偶可行性： $\beta_j \geq 0, \quad j = 1, 2, \dots, n$
- 互补松弛： $\beta_j g_j(\mathbf{x}) = 0, \quad j = 1, 2, \dots, n$

间隔与支持向量

超平面方程: $w^\top x + b = 0$



支持向量机基本型

- 最大间隔：寻找参数 w 和 b ，使得 γ 最大。

$$\begin{aligned} \arg \max_{w,b} \quad & \frac{2}{\|w\|} \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$

- 为了最大化间隔，仅需最大化 $\frac{2}{\|w\|}$ ，相当于最小化 $\|w\|^2$ 。


$$\begin{aligned} \arg \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \quad \quad \quad \text{\textcolor{red}{}w \text{ 的二次函数}} \quad \quad \quad \text{\textcolor{red}{}凸二次规划} \\ \text{s.t.} \quad & y_i(w^\top x_i + b) \geq 1, \quad i = 1, 2, \dots, m. \quad \text{\textcolor{red}{}w \text{ 的线性函数}} \end{aligned}$$

- 这就是支持向量机SVM的基本型。
- 可以使用SMO算法求解。

第6章 支持向量机

6.1 间隔与支持向量



6.2 对偶问题

6.3 核函数

6.4 软间隔与正则化

6.5 支持向量回归

6.6 核方法

对 偶

□ 某些使用拉格朗日获得的函数，使用求导的方法求解依然困难。

进而，需要对问题再进行一次转换，即使用一个数学技巧：**拉格朗日对偶**。

□ 所以在拉格朗日优化问题上，需要进行下面二个步骤：

- 将**有约束**的原始目标函数转换为**无约束**的新构造的拉格朗日目标函数
- 使用拉格朗日对偶性，将**不易求解**的优化问题转化为易求解的优化

对偶

第一步：将有约束的原始目标函数转换为无约束的拉格朗日函数

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

令：

$$\boldsymbol{\theta}(\mathbf{w}) = \max_{\alpha_i \geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

- **样本点在可行域外** $y_i (\mathbf{w}^T \mathbf{x}_i + b) < 1$ ：将 α_i 趋于无穷大，则 $\boldsymbol{\theta}(\mathbf{w})$ 趋于无穷大；
- **样本点在可行域内** $y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ ： $\boldsymbol{\theta}(\mathbf{w})$ 为原目标本身。

因此，可得新的目标函数为：

$$\boldsymbol{\theta}(\mathbf{w}) = \begin{cases} \frac{1}{2} \|\mathbf{w}\|^2, & \mathbf{x}_i \in \text{可行域} \\ +\infty & , \mathbf{x}_i \in \text{非可行域} \end{cases}$$

问题变为求新目标函数的最小值，即

$$\min_{\mathbf{w}, b} \max_{\alpha_i \geq 0} \mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$$

对偶

第二步：转换为**对偶问题**（即将不易求解的优化问题转换为易求解的问题）

$$\max_{\alpha_i \geq 0} \min_{\boldsymbol{w}, b} \mathcal{L}(\boldsymbol{w}, b, \boldsymbol{\alpha})$$

对偶问题求解

□ 拉格朗日乘子法

- 第一步：引入拉格朗日乘子 $\alpha_i \geq 0$ 得到拉格朗日函数

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i (y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1)$$

- 第二步：令 $\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha})$ 对 $\mathbf{w}$ 和 b 的偏导为零可得

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^m \alpha_i y_i = 0.$$

- 第三步：回代

$\mathbf{w}$ 由样本点及类别决定

对偶问题求解

$$\begin{aligned}\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^m \alpha_i [y_i (\mathbf{w}^T \mathbf{x}_i + b) - 1] \\&= \frac{1}{2} \mathbf{w}^T \mathbf{w} - \mathbf{w}^T \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i - b \sum_{i=1}^m \alpha_i y_i + \sum_{i=1}^m \alpha_i \\&= \frac{1}{2} \mathbf{w}^T \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i - \mathbf{w}^T \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i - b \cdot 0 + \sum_{i=1}^m \alpha_i \\&= \sum_{i=1}^m \alpha_i - \frac{1}{2} \left(\sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \right)^T \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \\&= \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j\end{aligned}$$

对偶问题求解

内侧的最小值求解完成后，求解外侧的最大值，从上式可得：

$$\max_{\alpha} \frac{1}{2} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$s.t. \alpha_i \geq 0, i = 1, 2, \dots, m$$

$$\sum_{i=1}^m \alpha_i y_i = 0$$

转换为等价问题：

$$\min_{\alpha} \frac{1}{2} \sum_{i,j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \frac{1}{2} \sum_{i=1}^m \alpha_i$$

$$s.t. \alpha_i \geq 0, i = 1, 2, \dots, m$$

$$\sum_{i=1}^m \alpha_i y_i = 0$$

解得最优解： $\alpha^* = (\alpha_1^*, \alpha_2^*, \dots, \alpha_m^*)^T$

计算可得：

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i \quad b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i^T \mathbf{x}_j$$

求得超平面： $\mathbf{w}^* \mathbf{x} + b^* = 0$

对偶问题求解

- 以上问题有更高效率的优化算法求解，即序列最小优化（SMO）算法。通过这个优化算法能得到 α ，再根据 α ，就可以求解出 w 和 b ，进而求得最初的目的：找到超平面，即"决策平面"。

$$wx+b=0$$

- 总结：为什么费力进行推导？因为要将最初的原始问题，转换到可以使用SMO算法求解的问题。

解的稀疏性

□ 最终模型:

$$f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i^\top \mathbf{x} + b$$

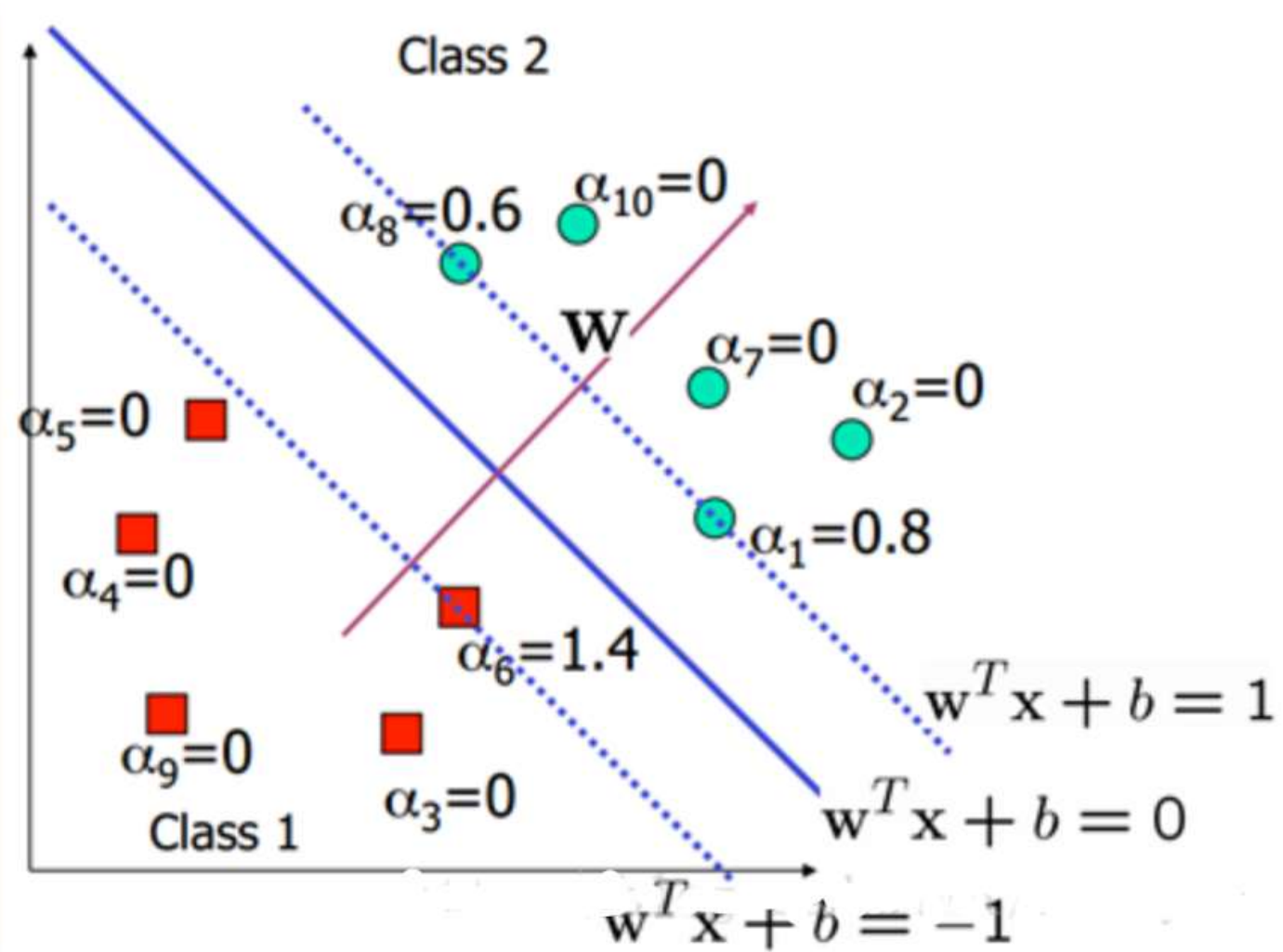
□ KKT条件: (满足此条件才能使用拉格朗日乘子法)

$$\begin{cases} \alpha_i \geq 0, \\ y_i f(\mathbf{x}_i) \geq 1, \\ \alpha_i (y_i f(\mathbf{x}_i) - 1) = 0. \end{cases}$$

$y_i f(\mathbf{x}_i) > 1 \quad \Rightarrow \quad \alpha_i = 0$ 该样本对 $f(\mathbf{x})$ 没有影响

$y_i f(\mathbf{x}_i) - 1 = 0 \quad \Rightarrow \quad$ 该样本位于边界, 为支持向量

支持向量机解的**稀疏性**: 训练完成后, 大部分的训练样本都不需保留, 最终模型仅与支持向量有关.



SMO算法

- 1996年, John Platt发布SMO算法, 用于训练SVM
- SMO表示序列最小化 (Sequential Minimal Optimization)。SMO算法是将大优化问题分解为多个小优化问题来求解。这些小优化问题往往很容易求解, 并且对它们进行顺序求解的结果与将它们作为整体来求解的结果完全一致的。在结果完全相同的同时, SMO算法的求解时间短很多。
- SMO算法的目标是求出一系列 α 和 b , 一旦求出了这些 α , 就很容易计算出权重向量 w 并得到分隔超平面。

求解方法 - SMO

□ 基本思路：不断执行如下两个步骤直至收敛.

- 第一步：选取一对需更新的变量 α_i 和 α_j .
- 第二步：固定 α_i 和 α_j 以外的参数，求解对偶问题更新 α_i 和 α_j .

□ 仅考虑 α_i 和 α_j 时，对偶问题的约束变为

$$\alpha_i y_i + \alpha_j y_j = - \sum_{k \neq i, j} \alpha_k y_k, \quad \alpha_i \geq 0, \quad \alpha_j \geq 0.$$

用一个变量表示另一个变量，回代入对偶问题可得一个单变量的二次规划，该问题具有闭式解.

□ 偏移项 b ：通过支持向量来确定.

第6章 支持向量机

6.1 间隔与支持向量

6.2 对偶问题

➡ 6.3 核函数

6.4 软间隔与正则化

6.5 支持向量回归

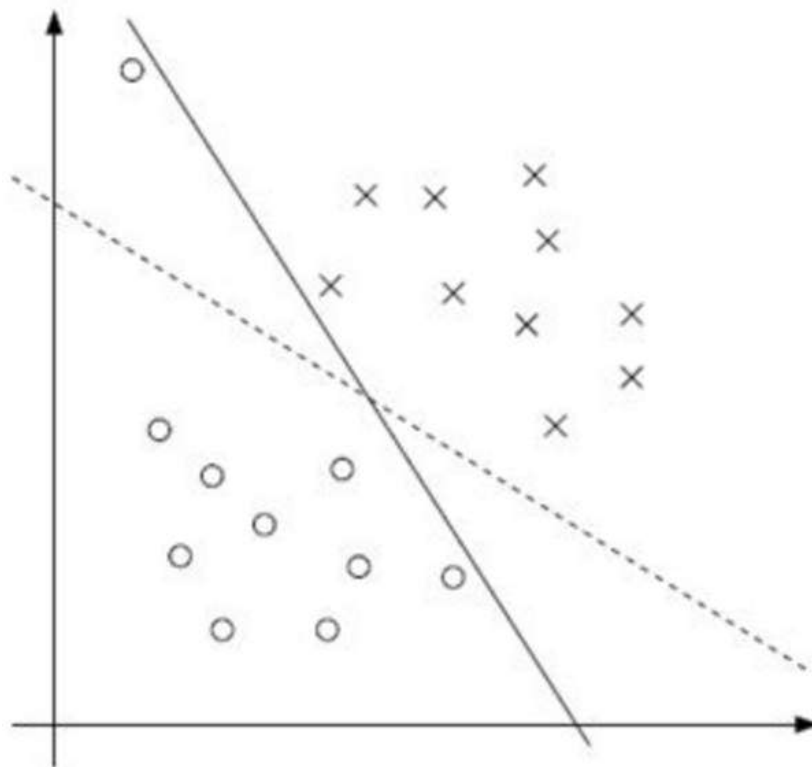
6.6 核方法

SVM类别

- SVM是一种二分类模型，处理的数据可以分为三类：
 - ✓ 线性可分，通过硬间隔最大化，学习线性分类器。
 - ✓ 近似线性可分，通过软间隔最大化，学习线性分类器
 - ✓ 线性不可分，通过核函数以及软间隔最大化，学习非线性分类器
- 线性分类器，在平面上对应直线；非线性分类器，在平面上对应曲线。
- 硬间隔对应于线性可分数据集，可以将所有样本正确分类，也正因为如此，受噪声样本影响很大，不推荐。
- 软间隔对应于通常情况下的数据集（近似线性可分或线性不可分），允许一些超平面附近的样本被错误分类，从而提升了泛化性能。松弛变量调节错分率。

SVM类别

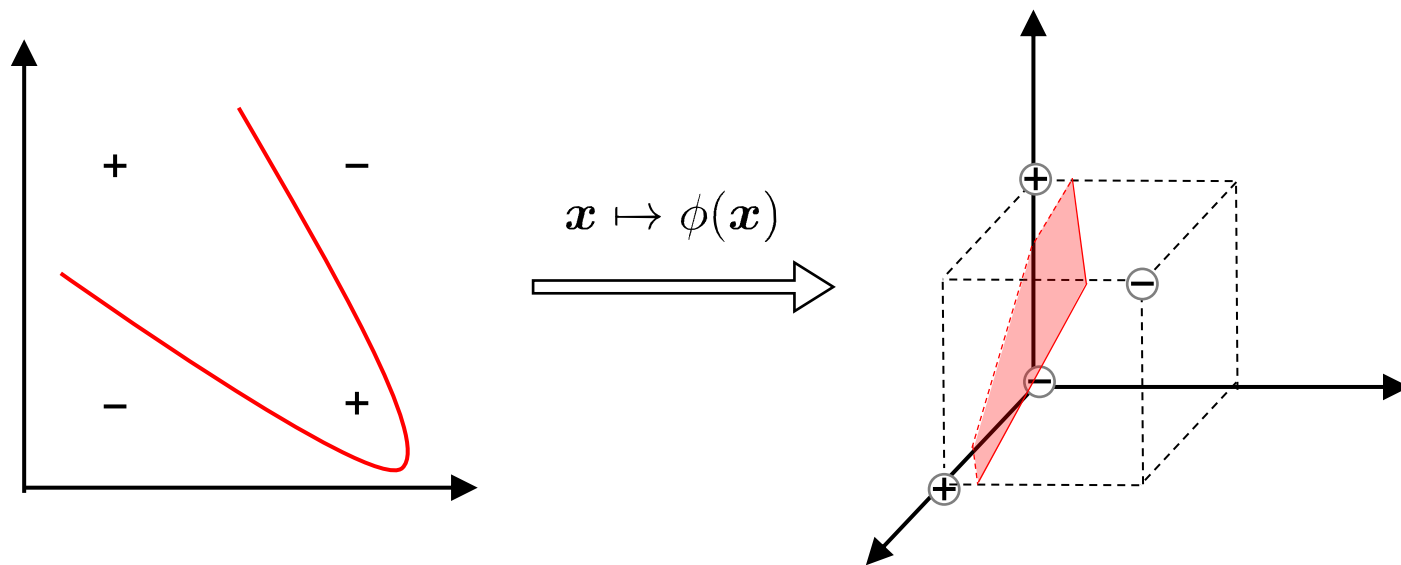
- 实线是由硬间隔最大化得到的，预测能力显然不及由软间隔最大化得到的虚线。



线性不可分

-Q: 若不存在一个能正确划分两类样本的超平面, 怎么办?

-A: 将样本从原始空间映射到一个更高维的特征空间, 使得样本在这个特征空间内线性可分.

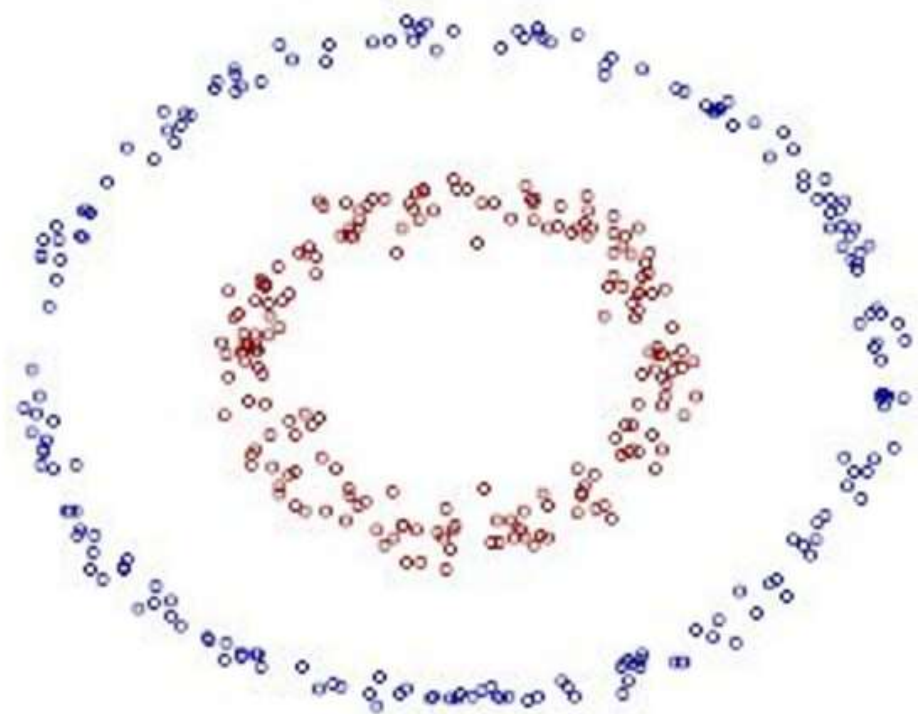


线性不可分

□ 直线不能很好的分开红点和蓝点。

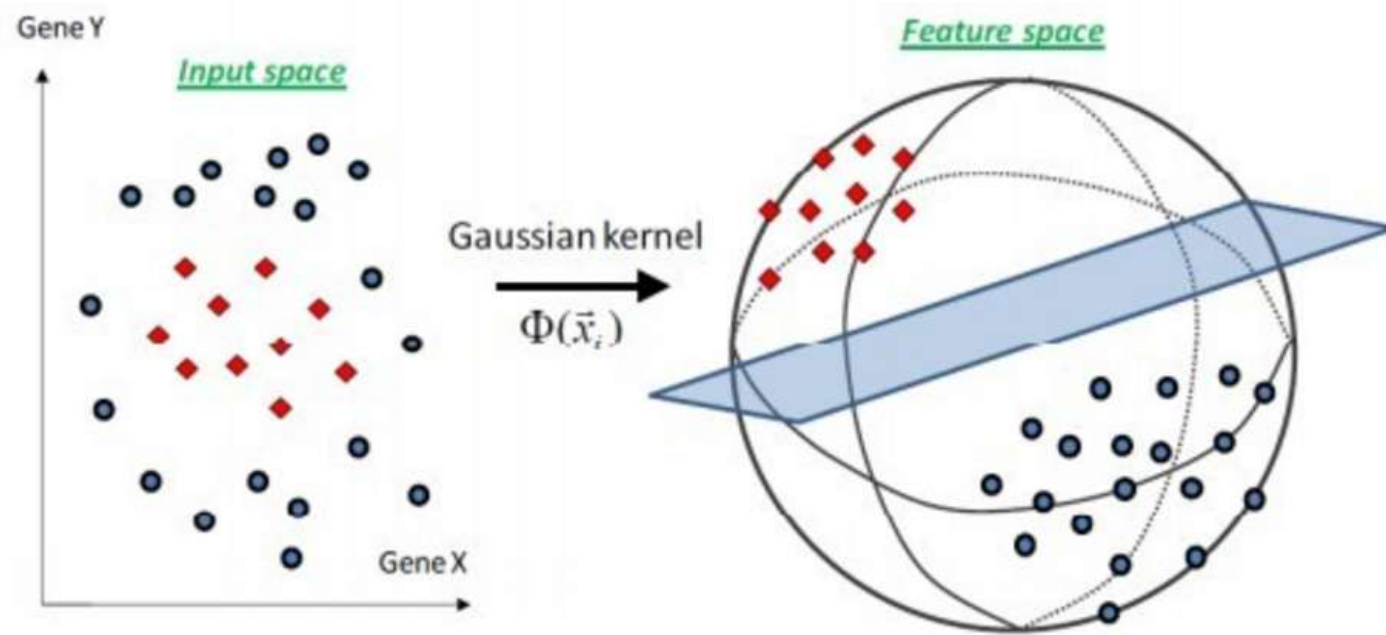
但是可以用一个介于红点与蓝点之间的类似圆的曲线将二者分开：

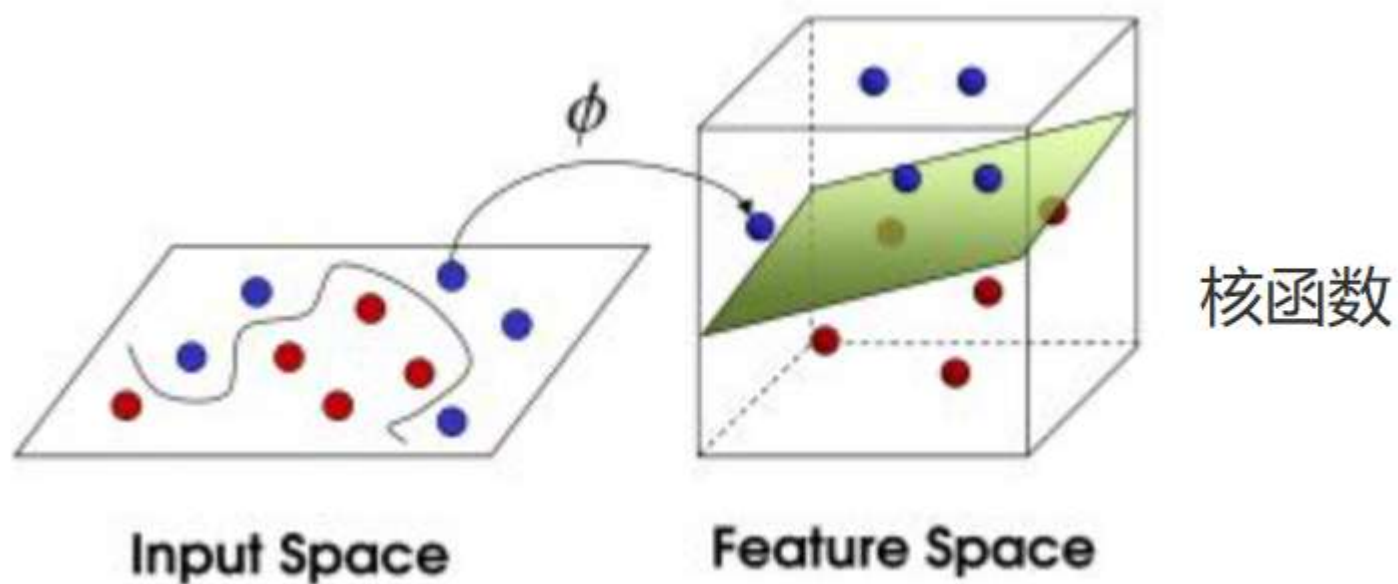
□ 假设这个黄色的曲线就是圆，不妨设其方程为 $x^2 + y^2 = 1$ 。将 x^2 映射为 X ， y^2 映射为 Y ，那么超平面变成了 $X+Y=1$ 。那么原空间的线性不可分问题，就变成了新空间的（近似）线性可分问题。核函数完成这一功能，实现从低维空间到高维空间的映射。



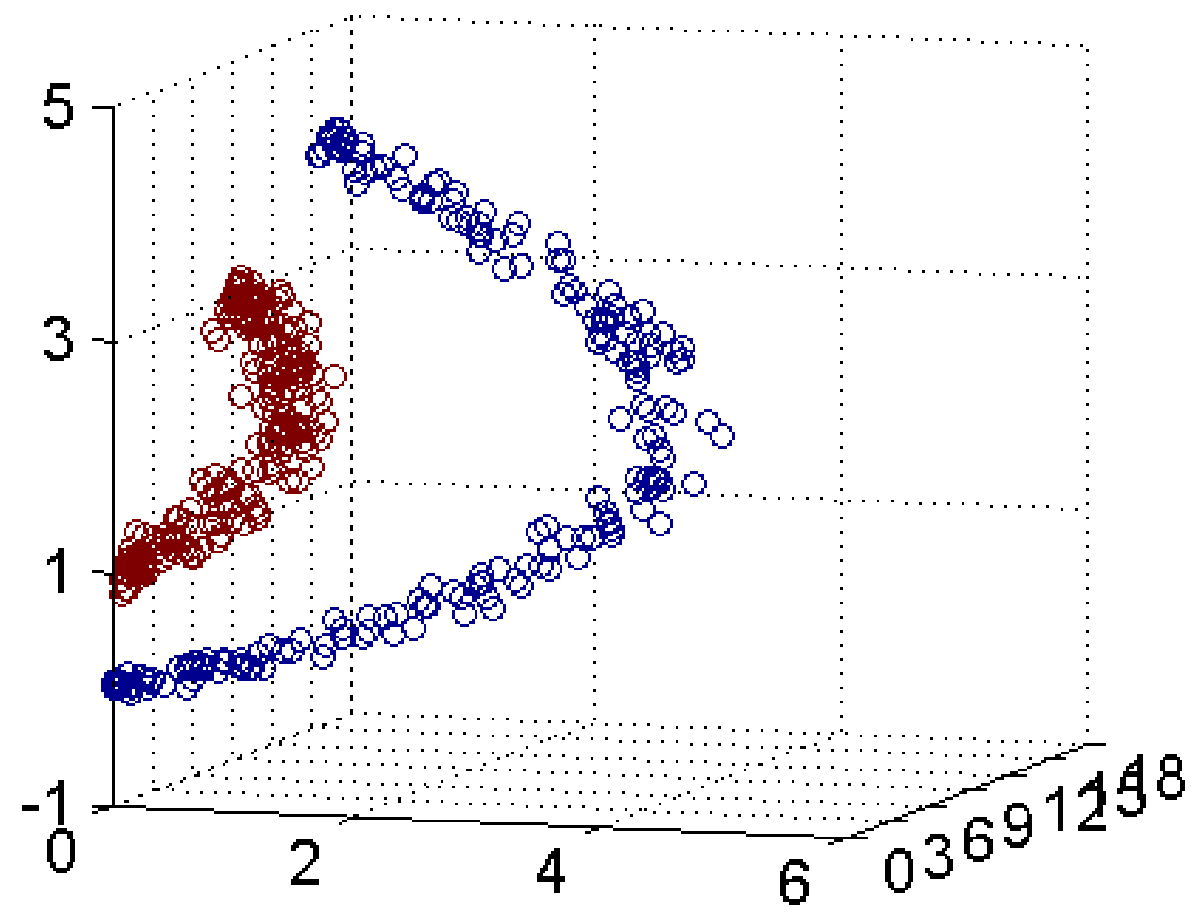
核函数

- 事实上大部分时候数据并不是线性可分的，这个时候满足这样条件的超平面就根本不存在。对于非线性的情况，SVM 的处理方法是选择一个核函数 $\kappa(\cdot, \cdot)$ ，通过**将数据映射到高维空间**，最终在高维特征空间中构造出最优分离超平面，从而把平面上本身不好分的非线性数据分开。来解决在原始空间中线性不可分的问题。





- 核函数的基本作用就是接受两个低维空间里的向量，能够计算出经过某个变换后在高维空间里的向量内积值。



核支持向量机

- 把线性不可分问题当成线性可分问题，参数一样

核支持向量机

□ 设样本 $\mathbf{x}$ 映射后的向量为 $\phi(\mathbf{x})$, 划分超平面 $f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b$ 为

原始问题

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$

对偶问题

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \boxed{\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)} - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \end{aligned}$$

只以内积的形式出现

预测

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m \alpha_i y_i \boxed{\phi(\mathbf{x}_i)^\top \phi(\mathbf{x})} + b$$

把线性不可分问题当成线性可分问题，参数一样

核函数

□ 基本想法：不显式地设计核映射，而是设计核函数.

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

□ Mercer定理(充分非必要)：只要一个对称函数所对应的核矩阵半正定，则它就能作为核函数来使用.

□ 常用核函数：

名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\delta^2}\right)$	$\delta > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	$\tanh$ 为双曲正切函数, $\beta > 0, \theta < 0$

第6章 支持向量机

6.1 间隔与支持向量

6.2 对偶问题

6.3 核函数

➡ 6.4 软间隔与正则化

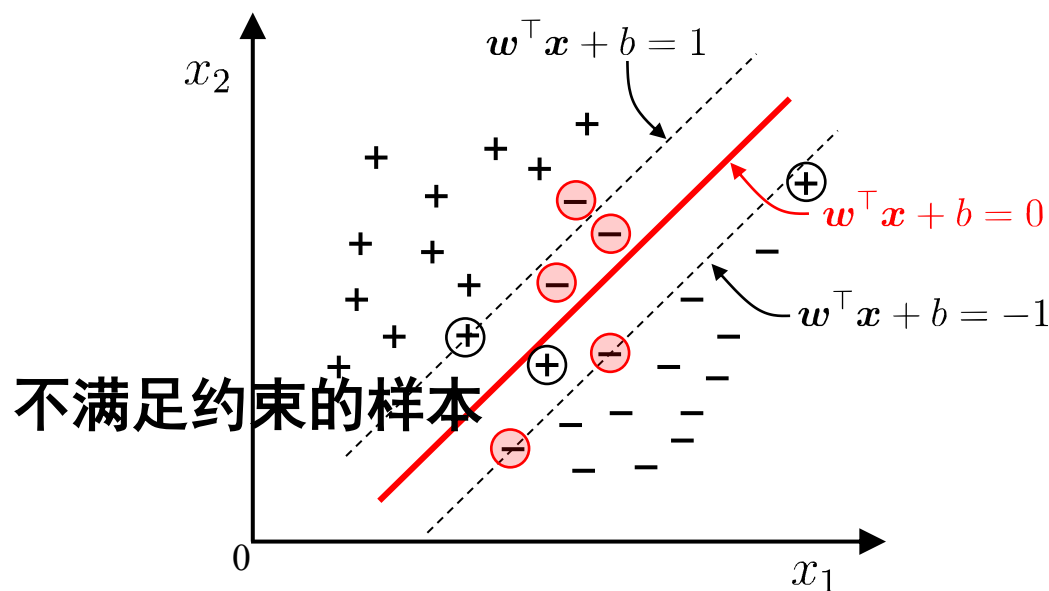
6.5 支持向量回归

6.6 核方法

软间隔

-Q: 现实中, 很难确定合适的核函数使得训练样本在特征空间中线性可分; 同时一个线性可分的结果也很难断定是否是有过拟合造成的.

-A: 引入”软间隔”的概念, 允许支持向量机在一些样本上不满足约束.



$$y_i(w^T x_i + b) \geq 1 - \xi_i$$

放弃对某些点（很大的可能是噪声）的精确分类，对分类器是损失，但是好处是分类面不必向这些点移动，得到更大的几何间隔（在低维空间中，分类边界更平滑）。需要权衡这种好处和损失。

0/1损失函数

- 基本想法：最大化间隔的同时，让不满足约束的样本应尽可能少。C是调和两者的系数，C值大时对误分类的惩罚增大，C值小时对误分类的惩罚减小

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m l_{0/1}(y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) - 1)$$

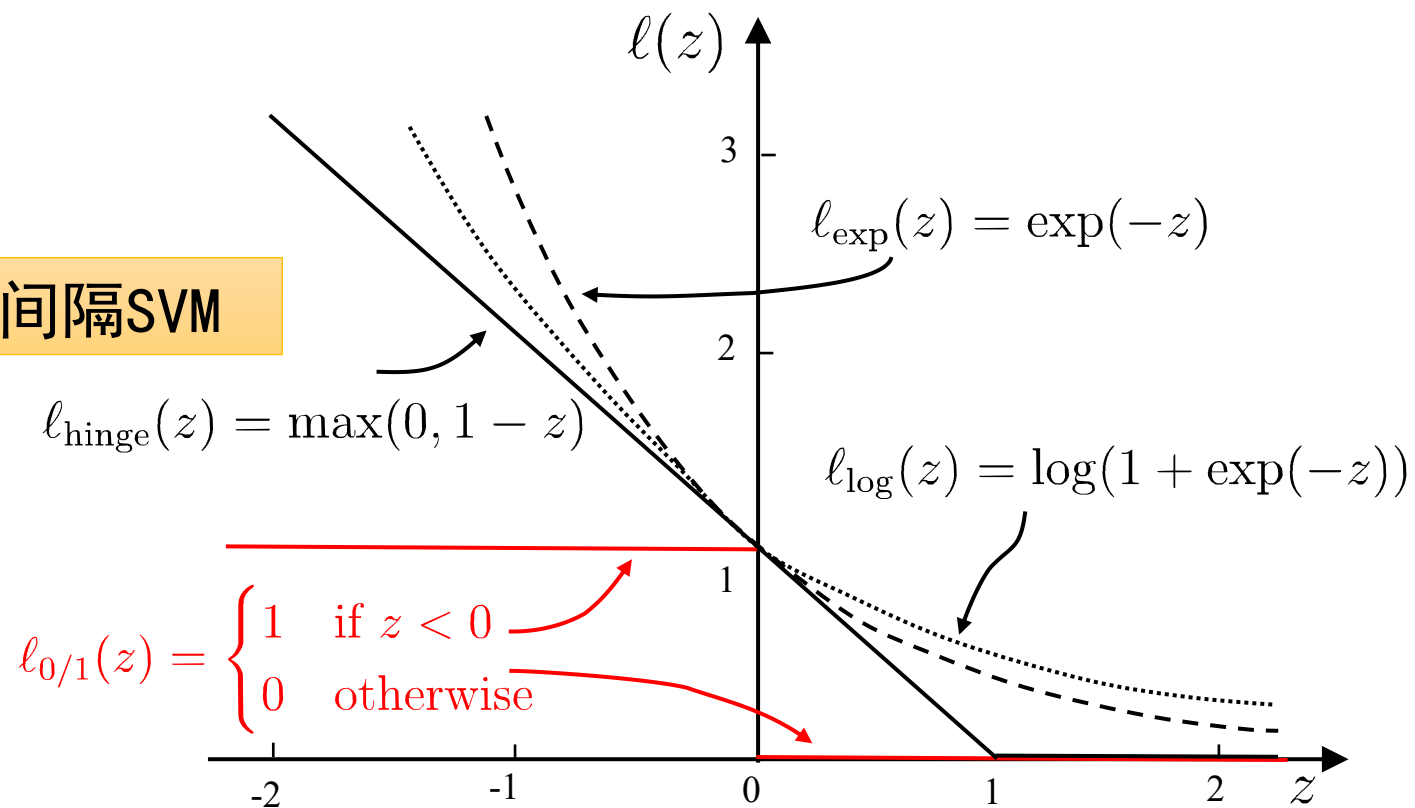
其中 $l_{0/1}$ 是“0/1损失函数”

$$l_{0/1} = \begin{cases} 1 & z < 0 \\ 0 & otherwise \end{cases}$$

- 存在的问题：0/1损失函数非凸、非连续，不易优化！

替代损失

软间隔SVM



替代损失函数数学性质较好，一般是0/1损失函数的上界

软间隔支持向量机

原始问题

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b))$$

对偶问题

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, m. \end{aligned}$$

根据KKT条件可推得最终模型仅与支持向量有关，也即hinge损失函数依然保持了支持向量机解的稀疏性。

正则化

- 支持向量机器学习模型的更一般形式

$$\min_f \Omega(f) + C \sum_{i=1}^m l(f(\mathbf{x}_i), y_i)$$



结构风险，描述模型的某些性质



经验风险，描述模型与训练数据的契合程度

- 通过替换上面两个部分，可以得到许多其他学习模型
 - 对数几率回归 (Logistic Regression)
 - 最小绝对收缩选择算子 (LASSO)
 -

第6章 支持向量机

6.1 间隔与支持向量

6.2 对偶问题

6.3 核函数

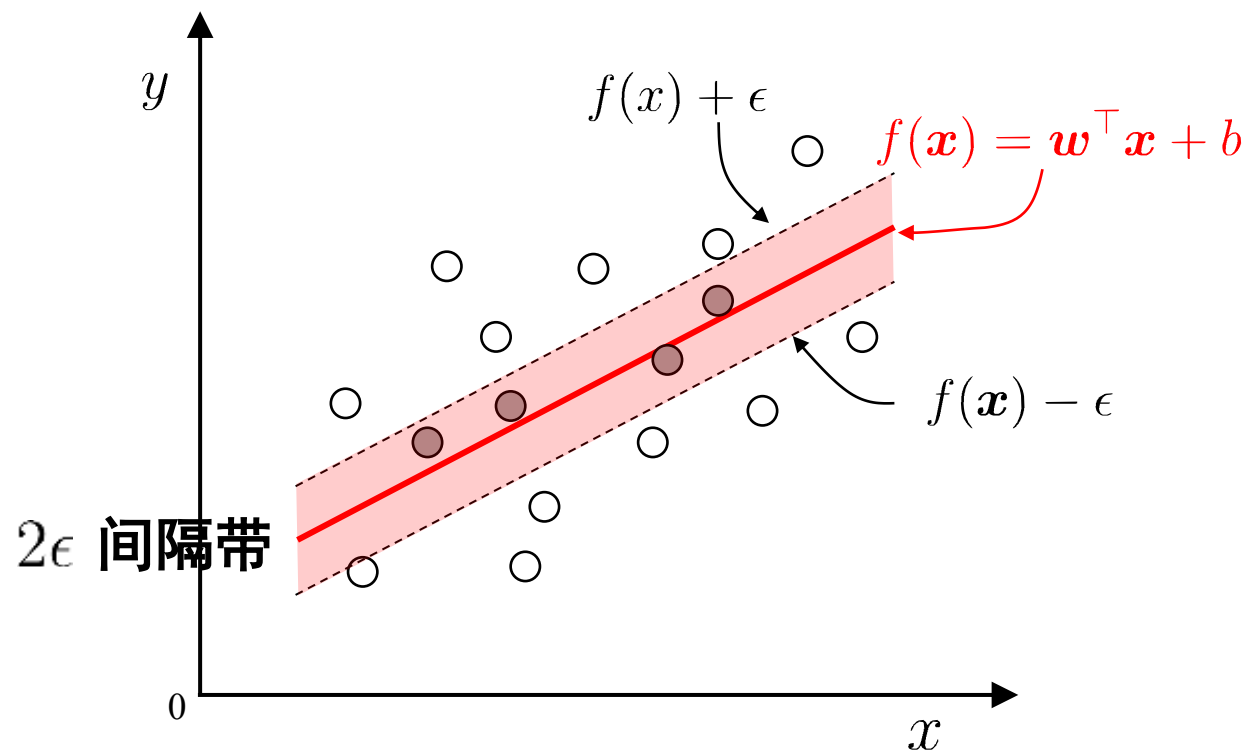
6.4 软间隔与正则化

➡ 6.5 支持向量回归

6.6 核方法

支持向量回归

特点：允许模型输出和实际输出间存在 的偏差. 2ϵ



支持向量回归 (SVR)

落入中间 2ϵ 间隔带的样本不计算损失，从而使得模型获得稀疏性.

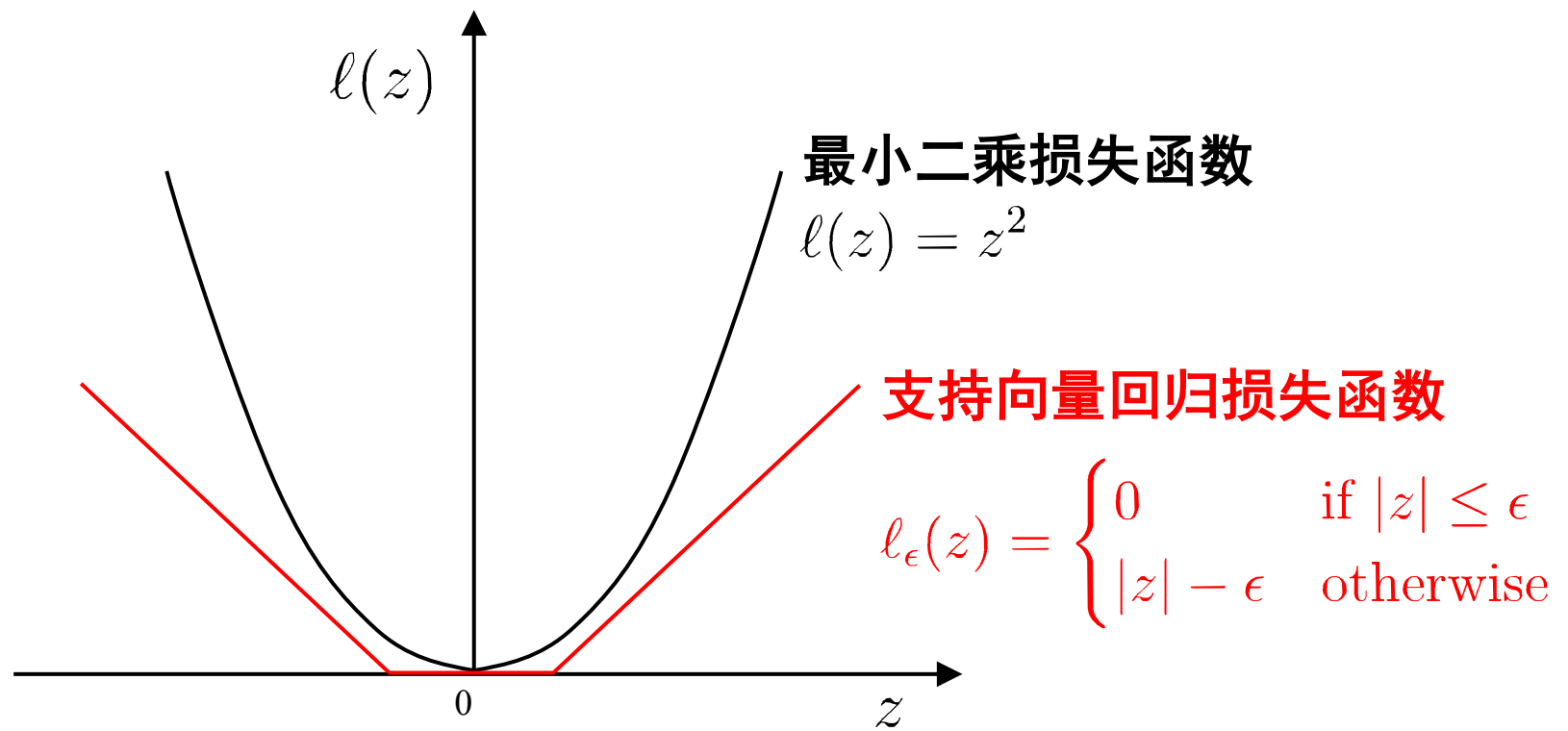
SVR问题可形式化为

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \ell_{\epsilon}(f(x_i) - y_i)$$

其中 C 为正则化常数, ℓ_{ϵ} 是图6.7所示的 ϵ -不敏感损失 (ϵ -insensitive loss)

$$\ell_{\epsilon}(z) = \begin{cases} 0, & \text{if } |z| \leq \epsilon; \\ |z| - \epsilon, & \text{otherwise.} \end{cases}$$

损失函数



形式化

原始问题

$$\begin{aligned} \min_{\mathbf{w}, b, \xi_i, \hat{\xi}_i} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m (\xi_i + \hat{\xi}_i) \\ \text{s.t.} \quad & y_i - \mathbf{w}^\top \phi(\mathbf{x}_i) - b \leq \epsilon + \xi_i, \\ & y_i - \mathbf{w}^\top \phi(\mathbf{x}_i) - b \geq -\epsilon - \hat{\xi}_i, \\ & \xi_i \geq 0, \hat{\xi}_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

对偶问题

$$\begin{aligned} \min_{\alpha, \hat{\alpha}} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (\alpha_i - \hat{\alpha}_i)(\alpha_j - \hat{\alpha}_j) \kappa(\mathbf{x}_i, \mathbf{x}_j) + \sum_{i=1}^m (\alpha_i(\epsilon - y_i) + \hat{\alpha}_i(\epsilon + y_i)) \\ \text{s.t.} \quad & \sum_{i=1}^m (\alpha_i - \hat{\alpha}_i) = 0, \\ & 0 \leq \alpha_i \leq C, \quad 0 \leq \hat{\alpha}_i \leq C. \end{aligned}$$

预测

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m (\hat{\alpha}_i - \alpha_i) y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b$$

第6章 支持向量机

6.1 间隔与支持向量

6.2 对偶问题

6.3 核函数

6.4 软间隔与正则化

6.5 支持向量回归

➡ 6.6 核方法

表示定理

支持向量机 $f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m \alpha_i y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b$

支持向量回归 $f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m (\hat{\alpha}_i - \alpha_i) y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b$

结论：无论是支持向量机还是支持向量回归，学得的模型总可以表示成核函数的线性组合。

更一般的结论(表示定理)：对于任意单调增函数 Ω 和任意非负损失函数 l ，优化问题

$$\min_{h \in \mathbb{H}} F(h) = \Omega(\|h\|_{\mathbb{H}}) + l(h(\mathbf{x}_1), \dots, h(\mathbf{x}_m))$$

令 $\mathbb{H}$ 为核函数 κ 对应的再生核希尔伯特空间

的解总可以写为

$$h^*(\mathbf{x}) = \sum_{i=1}^m \alpha_i \kappa(\mathbf{x}, \mathbf{x}_i)$$

核线性判别分析

□ 通过表示定理可以得到很多线性模型的”核化”版本

- 核SVM
- 核LDA
- 核PCA
-

□ 核LDA：先将样本映射到高维特征空间，然后在此特征空间中做线性判别分析

$$\begin{array}{c} \max_w J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_b^\phi \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_w^\phi \mathbf{w}} \\ \downarrow \\ \max_{\boldsymbol{\alpha}} J(\boldsymbol{\alpha}) = \frac{\boldsymbol{\alpha}^\top \mathbf{M} \boldsymbol{\alpha}}{\boldsymbol{\alpha}^\top \mathbf{N} \boldsymbol{\alpha}} \end{array} \quad h(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) = \sum_{i=1}^m \alpha_i \kappa(\mathbf{x}_i, \mathbf{x})$$

Take Home Message

- 支持向量机的”最大间隔”思想
- 对偶问题及其解的稀疏性
- 通过向高维空间映射解决线性不可分的问题
- 引入”软间隔”缓解特征空间中线性不可分的问题
- 将支持向量的思想应用到回归问题上得到支持向量回归
- 将核方法推广到其他学习模型

sklearn中的SVM

- ❑ Sklearn包含三个SVM分类器:
- ❑ `sklearn.svm.LinearSVC`
- ❑ `sklearn.svm.SVC`
- ❑ `sklearn.svm.NuSVC`

SVC、LinearSVC与NuSVC的区别与选型建议

分类器	核心功能对比	性能与效率权衡	适用场景划分	参数设计差异
SVC	支持多核函数，适合中小数据集	SVC使用libsvm，支持核技巧但较慢，适用于小样本。	非线性问题首选SVC配合RBF核	使用C调节正则强度
LinearSVC	仅线性核，优化更快	基于liblinear，训练高效，适合大规模线性问题	高维稀疏数据如文本分类可用LinearSVC	使用C调节正则强度
NuSVC	以参数 ν 控制支持向量比例，语义更直观	SVC使用libsvm，支持核技巧但较慢，适用于小样本。	需精细控制模型复杂度时考虑NuSVC	NuSVC用 $\nu \in (0, 1)$ 替代C，直接约束误分点和支持向量占比，物理意义更明确

sklearn中的SVM

sklearn中的SVM

```
class sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True
```

C:

C-SVC的惩罚参数C，也就是优化函数中松弛变量的系数，默认值是1.0。C越大，相当于惩罚松弛力度越强，（

kernel:

核函数，默认是rbf，可以是'linear'，'poly'，'rbf'，'sigmoid'，'precomputed'

0 - 线性: $u \cdot v$

1 - 多项式: $(\gamma \cdot u \cdot v + \text{coef0})^{\text{degree}}$

2 - RBF函数: $\exp(-\gamma |u-v|^2)$

3 -sigmoid: $\tanh(\gamma \cdot u \cdot v + \text{coef0})$

degree : 多项式poly函数的维度，默认是3，选择其他核函数时会被忽略。

gamma:

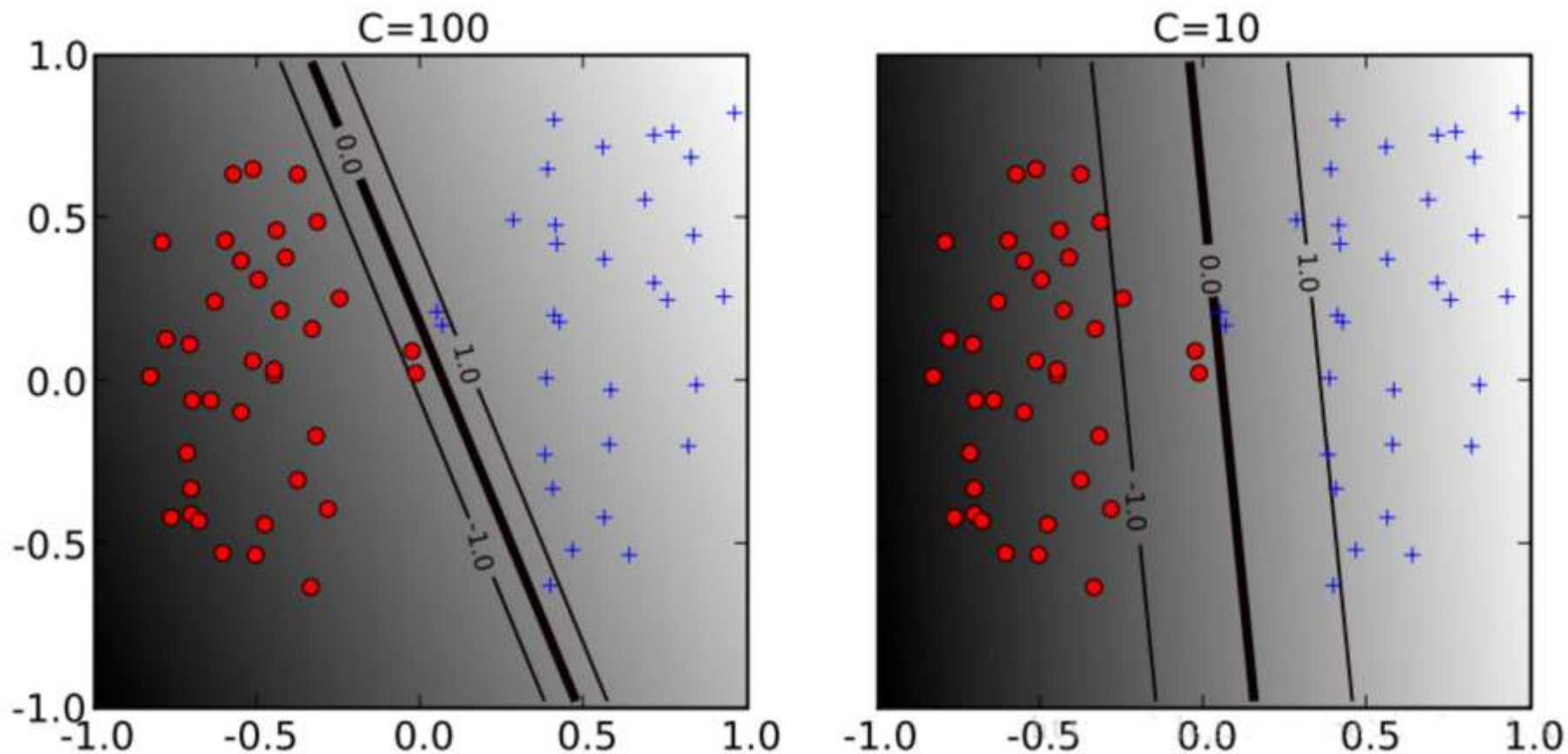
'rbf'，'poly' 和 'sigmoid'的核函数参数。默认是'auto'，则会选择 $1/n_features$

coef0 :

核函数的常数项。对于'poly'和 'sigmoid'有用。

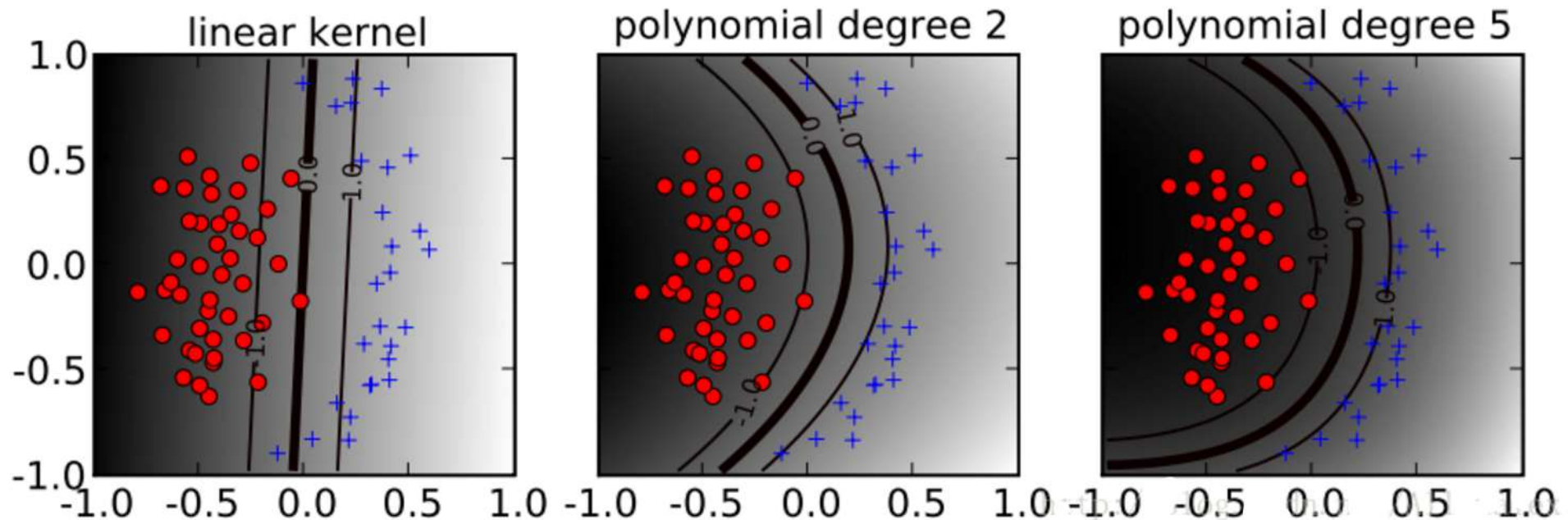
sklearn中的SVM

- 惩罚因子 C 越大，越要尽可能的考虑错误点(也就是说，尽量不要分错)，导致分界距离也变小。坏处是若干错误的“离群点”会对分界面的位置方向影响很大。



sklearn中的SVM

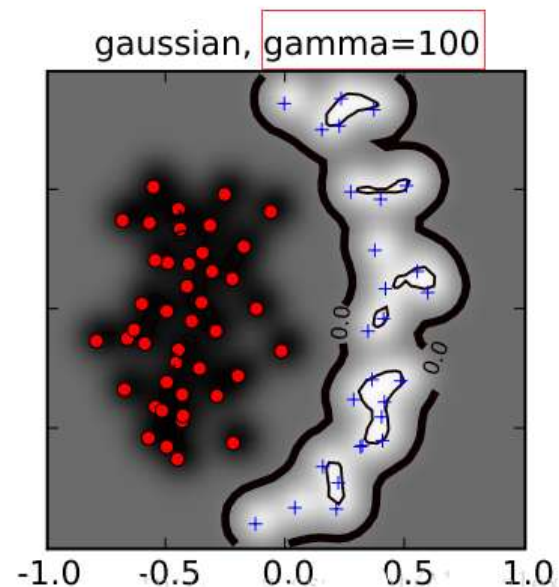
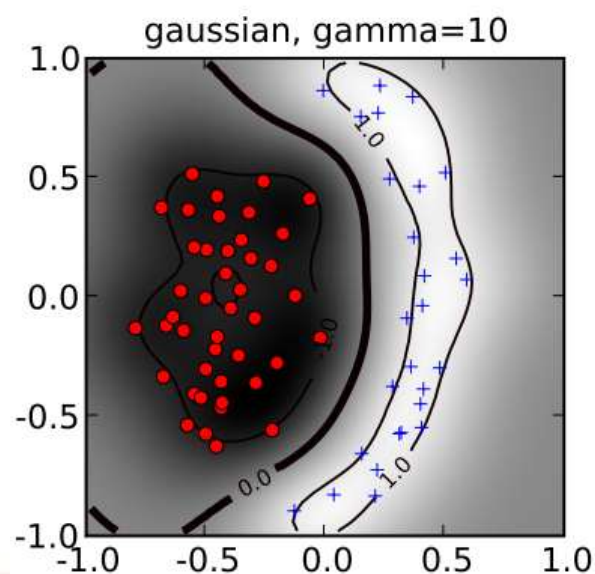
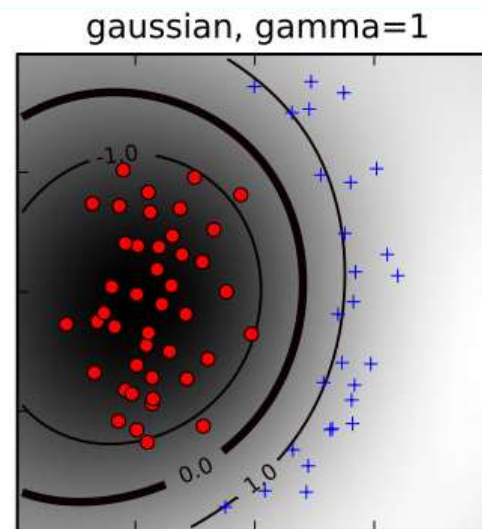
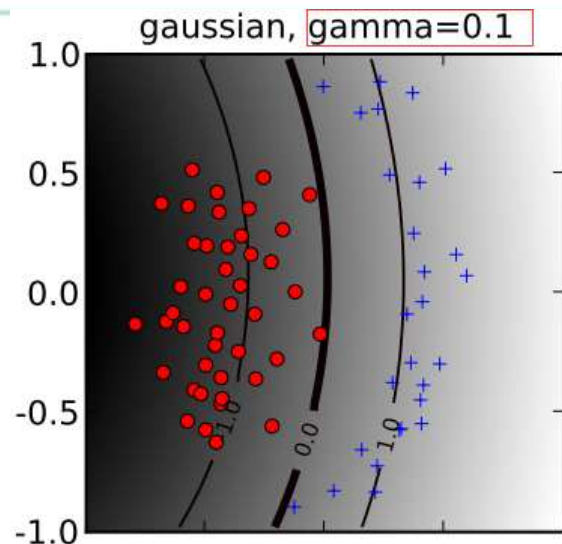
□ **degree**越大，分类器越灵活。太大会出现过拟合。



sklearn中的SVM

□ **gamma**

越大，
分类器
越灵活。
太大会
出现过
拟合。



```
from sklearn import svm,datasets
from sklearn.model_selection import train_test_split
from sklearn import metrics
#调用SVC()
#predict() 预测
iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y,\
                                                    test_size=0.30, random_state=1)

clf = svm.SVC()
clf.fit(X_train, y_train)    SVM model accuracy(in %): 97.7777777778
y_pred = clf.predict(X_test)

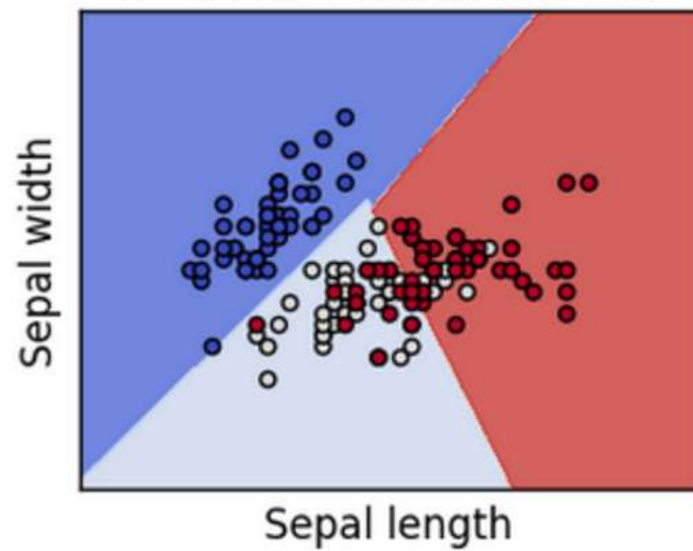
print("SVM model accuracy(in %):", \
      metrics.accuracy_score(y_test, y_pred)*100)
```

核的选择

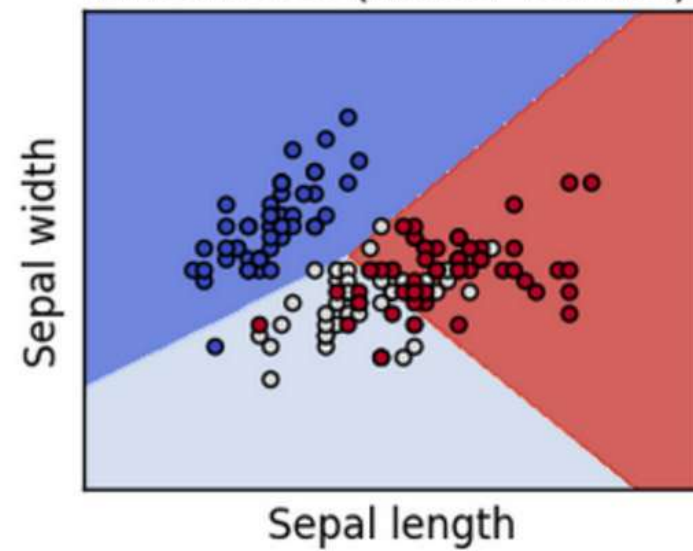
- 在不同数据集上，不同核的表现是差异较大的。

```
clf_linear = svm.SVC(kernel='linear').fit(x, y)
#clf_linear = svm.LinearSVC().fit(x, y)
clf_poly = svm.SVC(kernel='poly', degree=3).fit(x, y)
clf_rbf = svm.SVC().fit(x, y)
clf_sigmoid = svm.SVC(kernel='sigmoid').fit(x, y)
```

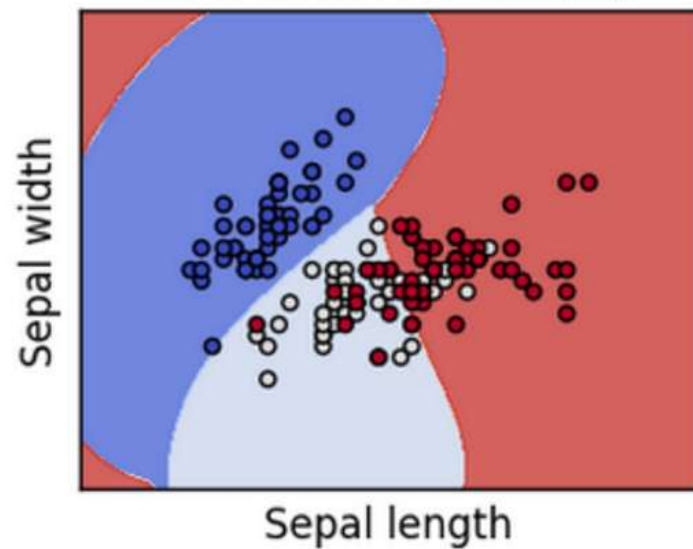
SVC with linear kernel



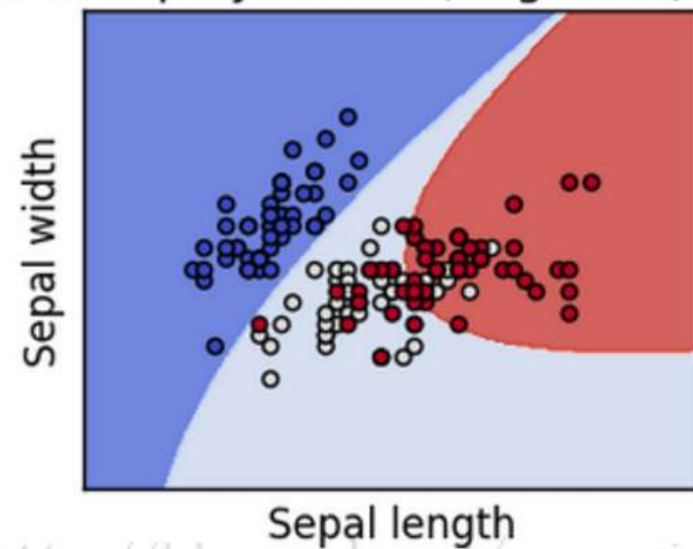
LinearSVC (linear kernel)



SVC with RBF kernel

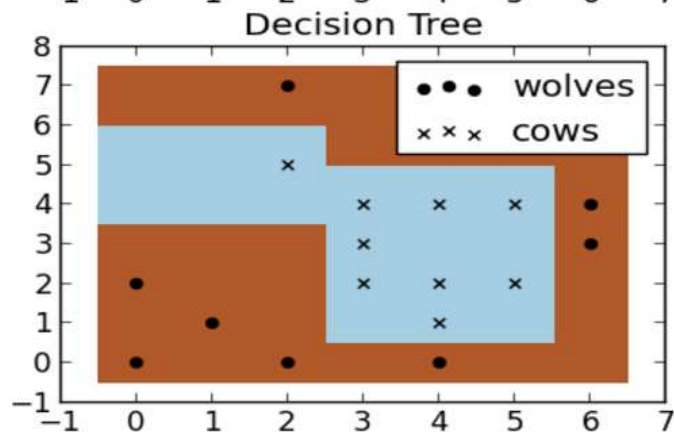
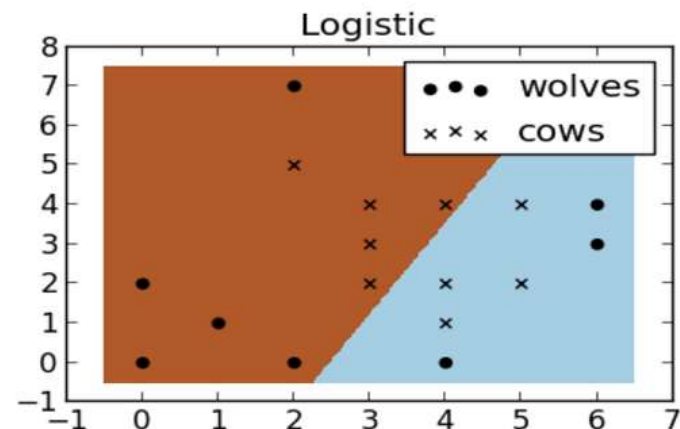
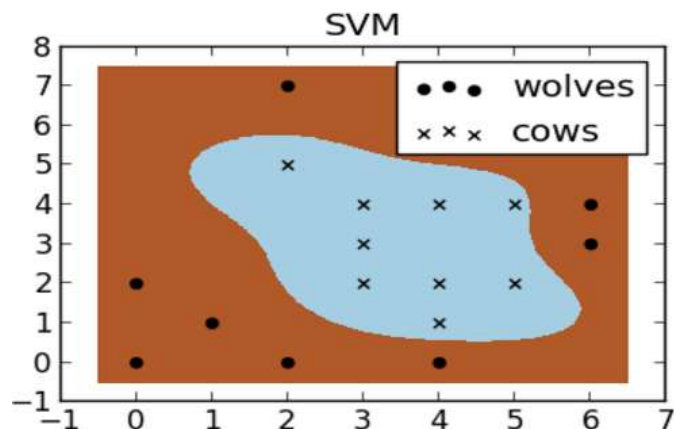


SVC with polynomial (degree 3) kernel



比较

□ 这个例子
单说明了
SVM使用非
线性分类器
的优点。



SVM特点

□ 优点：

- 在高维空间中有效。
- 在尺寸数量大于样本数量的情况下仍然有效。SVM解决问题的时候，和样本的维数是无关的（甚至样本是上万维的都可以，这使得SVM很适合用来解决文本分类的问题，当然，有这样的能力也因为引入了核函数）。
- 通用：可以为决策函数指定不同的内核函数。提供公共内核，但也可指定自定义内核。
- 与问题的复杂度比起来，SVM算法要求的样本数是相对比较少的。

□ 缺点：

如果特征的数目远大于样品的数目，则该方法可能给出差的性能。不适合过大数据量，噪声很多的数据。计算复杂度比较高。



重庆大学
CHONGQING UNIVERSITY

THANKS
祝学习快乐!